

Felipe Silvestre Cardoso Roberto
170425

Implementação do Compilador C-

São José dos Campos - Brasil
Junho de 2026

Felipe Silvestre Cardoso Roberto
170425

Implementação do Compilador C-

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina de Laboratório de Sistemas Computacionais: Compiladores.

Docente: Prof. Dr. Luiz Eduardo Galvão Martins.

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - Campus São José dos Campos

São José dos Campos - Brasil

Junho de 2026

Resumo

Este relatório descreve o desenvolvimento de um compilador para a linguagem C-, um subconjunto da linguagem C, tendo como arquitetura-alvo um processador autoral baseado na filosofia RISC e implementado em FPGA. O projeto compreende as etapas de análise léxica, análise sintática, análise semântica, geração de código intermediário, geração de código assembly e geração de código executável em formato binário.

A fase de análise realiza o reconhecimento dos tokens da linguagem, a verificação da estrutura gramatical do programa-fonte, a construção da árvore sintática abstrata e a validação de regras semânticas por meio da tabela de símbolos. A fase de síntese realiza a conversão da representação intermediária em quádruplas para instruções assembly compatíveis com o conjunto de instruções do processador, seguida da codificação dessas instruções para o formato binário utilizado pela memória de programa.

O relatório também apresenta o conjunto de instruções da máquina-alvo, a organização da memória, o uso de registradores, o gerenciamento da pilha, o tratamento de chamadas de função, recursão, variáveis, vetores e desvios de fluxo. A validação do compilador é realizada por meio de três programas de teste: cálculo do máximo divisor comum, fatorial recursivo e ordenação por seleção. Para cada exemplo, são apresentados o código-fonte, o código intermediário, o código assembly, o código executável e as correspondências entre essas representações.

Palavras-chave: Compiladores. Linguagem C-. Código intermediário. Assembly. RISC. FPGA.

Lista de ilustrações

Figura 1 – Diagrama de blocos do processador	11
Figura 2 – Datapath do processador	11
Figura 3 – Diagrama de atividades do ciclo de execução do processador	12
Figura 4 – Diagrama de blocos da fase de análise do compilador	19
Figura 5 – Diagrama de atividades da fase de análise do compilador	20
Figura 6 – Representação simplificada da AST gerada para o código-fonte de exemplo.	26
Figura 7 – Diagrama de blocos da fase de síntese do compilador	32
Figura 8 – Diagrama de atividades da fase de síntese do compilador	33

Lista de tabelas

Tabela 1 – Formatos das instruções implementadas	14
Tabela 2 – Instruções do Tipo R	15
Tabela 3 – Instruções do Tipo I	15
Tabela 4 – Instruções do Tipo J	16
Tabela 5 – Instruções do Tipo B	16
Tabela 6 – Instruções do Tipo N	16
Tabela 7 – Instruções do Tipo IO	17
Tabela 8 – Principais quádruplas geradas na fase de síntese	35
Tabela 9 – Mapeamento de quádruplas aritméticas, memória e controle	36
Tabela 10 – Mapeamento de quádruplas de função, contexto e entrada/saída	36
Tabela 11 – Formatos de codificação das instruções	37
Tabela 12 – Tabela completa de correspondência do Algoritmo de Euclides (MDC)	41
Tabela 13 – Tabela completa de correspondência do Algoritmo de Fatorial Recursivo	44
Tabela 14 – Tabela comparativa do Algoritmo de Selection Sort	48

Sumário

1	INTRODUÇÃO	8
2	O PROCESSADOR	10
2.1	Visão Geral da Arquitetura	10
2.2	Diagrama de Blocos do Processador	10
2.3	Ciclo de Execução do Processador	12
2.4	Componentes do Processador	13
2.4.1	Contador de Programa	13
2.4.2	Memória de Programa	13
2.4.3	Banco de Registradores	13
2.4.4	Unidade Lógica e Aritmética	13
2.4.5	Memória de Dados	13
2.4.6	Unidade de Controle	13
2.4.7	Módulo de Entrada e Saída	14
2.5	Conjunto de Instruções	14
2.5.1	Formatos das Instruções	14
2.5.2	Instruções do Tipo R	14
2.5.3	Instruções do Tipo I	15
2.5.4	Instruções do Tipo J	15
2.5.5	Instruções do Tipo B	16
2.5.6	Instruções do Tipo N	16
2.5.7	Instruções do Tipo IO	16
2.6	Organização da Memória	17
2.7	Relação com o Compilador	17
3	COMPILADOR: FASE DE ANÁLISE	18
3.1	Modelagem	18
3.1.1	Diagrama de Blocos (SysML)	19
3.1.2	Diagrama de Atividades (SysML)	20
3.1.3	Estruturas Internas	21
3.2	Análise Léxica	21
3.2.1	Definições Léxicas	21
3.2.2	Palavras-chave	21
3.2.3	Operadores e Delimitadores	22
3.2.4	Comentários, Números e Identificadores	22
3.2.5	Tratamento de Erros Léxicos	23

3.3	Análise Sintática	23
3.3.1	Declaração de Tokens e Tipos	23
3.3.2	Construção da AST	24
3.3.3	Precedência e Ambiguidade	25
3.3.4	Tratamento de Erros Sintáticos	25
3.3.5	Exemplo de AST	25
3.4	Tabela de Símbolos	26
3.4.1	Estrutura de Dados	26
3.4.2	Inserção e Busca	27
3.4.3	Símbolos Pré-definidos	27
3.5	Análise Semântica	28
3.5.1	Verificação de Declarações	29
3.5.2	Verificação de Funções	29
3.5.3	Verificação de Chamadas de Função	29
3.5.4	Verificação de Variáveis e Vetores	30
3.5.5	Verificação da Função Main	30
4	COMPILADOR: FASE DE SÍNTESE	31
4.1	Modelagem	31
4.1.1	Diagrama de Blocos (SysML)	32
4.1.2	Diagrama de Atividades (SysML)	33
4.1.3	Estruturas Auxiliares	34
4.2	Geração do Código Intermediário	34
4.2.1	Formato das Quádruplas	34
4.2.2	Tipos de Quádruplas Definidos	35
4.3	Geração do Código Assembly	35
4.4	Geração do Código Executável	37
4.5	Gerenciamento de Memória	38
4.5.1	Gerenciamento de Registradores	38
4.5.2	Pilha de Execução e Chamadas de Função	39
4.5.3	Vetores	39
5	EXEMPLOS DE USO DO COMPILADOR	40
5.1	Algoritmo de Euclides: Máximo Divisor Comum	40
5.1.1	Código-fonte	40
5.1.2	Tabela de correspondência	41
5.1.3	Análise comparativa	42
5.2	Algoritmo de Fatorial Recursivo	44
5.2.1	Código-fonte	44
5.2.2	Tabela de correspondência	44

5.2.3	Análise comparativa	46
5.3	Algoritmo de Ordenação: Selection Sort	47
5.3.1	Código-fonte	47
5.3.2	Tabela de correspondência	48
5.3.3	Análise comparativa	53
5.4	Síntese dos Resultados	54
6	CONCLUSÃO	55
6.1	Dificuldades Encontradas	55
6.2	Destaques	56
6.3	Trabalhos Futuros	56
	REFERÊNCIAS	58

1 Introdução

Compiladores são sistemas responsáveis por traduzir programas escritos em uma linguagem-fonte para uma linguagem-alvo, preservando o comportamento especificado pelo código original (LOUDEN; SILVA, 2004; AHO et al., 2007). Esse processo envolve etapas de análise e síntese, divisão clássica apresentada na literatura de compiladores (AHO et al., 2007). Na fase de análise, o programa-fonte é decomposto em unidades léxicas, validado segundo uma gramática e verificado semanticamente. Na fase de síntese, as representações internas produzidas durante a análise são convertidas em código intermediário, código assembly e, posteriormente, código binário para uma arquitetura-alvo.

No contexto deste trabalho, a linguagem-fonte adotada é a linguagem C-, um subconjunto da linguagem C utilizado na disciplina de Laboratório de Sistemas Computacionais: Compiladores. A linguagem contempla construções como declarações de variáveis, vetores, funções, comandos condicionais, comandos de repetição, chamadas de função, retorno de valores, operações aritméticas, operações relacionais e comandos de entrada e saída. A linguagem-alvo corresponde ao conjunto de instruções de um processador autoral com conjunto de instruções customizado, baseado em princípios RISC, implementado em Verilog e utilizado como arquitetura-alvo neste trabalho.

O objetivo deste projeto é implementar um compilador para a linguagem C- capaz de gerar código binário compatível com a memória de instruções do processador desenvolvido. Para isso, o compilador realiza a análise léxica do programa-fonte por meio do reconhecimento de tokens, a análise sintática por meio da aplicação das regras gramaticais da linguagem, a construção da árvore sintática abstrata, a análise semântica com uso de tabela de símbolos, a geração de código intermediário em quádruplas, a tradução dessas quádruplas para código assembly e a codificação final das instruções em formato binário.

A integração entre o compilador e o processador exige a definição de uma correspondência entre as construções da linguagem C- e as instruções disponíveis na arquitetura-alvo. Dessa forma, comandos de atribuição, expressões aritméticas, desvios condicionais, laços, chamadas de função, recursão, acesso a vetores e operações de entrada e saída devem ser convertidos para sequências de instruções compatíveis com o conjunto de instruções implementado. Além disso, o compilador deve tratar o uso de registradores, a organização da memória, o cálculo de endereços, a resolução de rótulos e o gerenciamento da pilha de execução.

Este relatório está organizado da seguinte forma. O Capítulo 2 apresenta o processador utilizado como arquitetura-alvo, incluindo o diagrama de blocos, os componentes internos, o conjunto de instruções e a organização da memória. O Capítulo 3 descreve a

fase de análise do compilador, abordando a modelagem, a análise léxica, a análise sintática, a tabela de símbolos e a análise semântica. O Capítulo 4 apresenta a fase de síntese, incluindo a geração de código intermediário, a geração de código assembly, a geração do código binário e o gerenciamento de memória. O Capítulo 5 apresenta exemplos de uso do compilador, contendo o código-fonte, o código intermediário, o código assembly, o código binário e as correspondências entre essas representações. Por fim, o Capítulo 6 apresenta a conclusão, incluindo dificuldades encontradas, destaques do projeto e trabalhos futuros.

2 O Processador

Este capítulo apresenta o processador utilizado como arquitetura-alvo do compilador desenvolvido. São descritos a organização geral do processador, seus principais componentes, o conjunto de instruções implementado e a organização da memória. Essas informações definem as restrições utilizadas pela fase de síntese do compilador durante a geração de código assembly e de código executável.

2.1 Visão Geral da Arquitetura

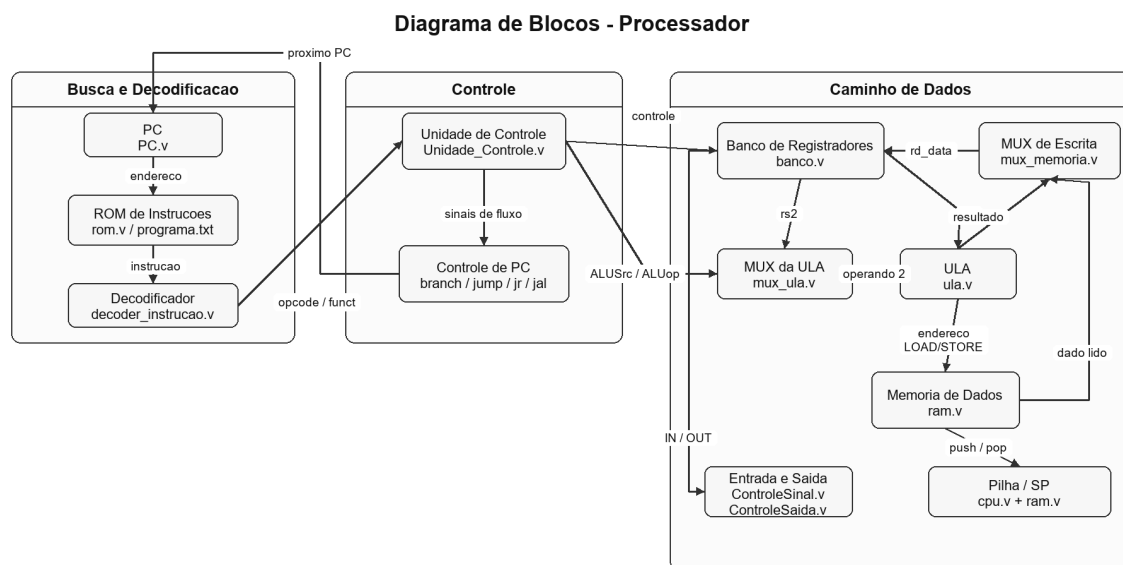
O processador utilizado neste trabalho segue a filosofia RISC ([PATTERSON; WATERMAN, 2019](#)), com instruções de tamanho fixo de 32 bits e execução em modelo monociclo. A arquitetura possui memória de instruções e memória de dados separadas, seguindo uma organização do tipo Harvard ([PATTERSON; HENNESSY, 2017](#)). Essa separação permite que o código executável gerado pelo compilador seja armazenado na memória de programa, enquanto variáveis, vetores, parâmetros e dados temporários são manipulados na memória de dados.

O conjunto de instruções foi definido para suportar as operações necessárias à tradução de programas escritos em C-. Entre essas operações estão instruções aritméticas, lógicas, acesso à memória, desvios condicionais, saltos, chamadas de função, retorno de função e entrada e saída. A geração de código do compilador considera diretamente os formatos de instrução, os campos de registradores, os imediatos, os rótulos e a organização da pilha.

2.2 Diagrama de Blocos do Processador

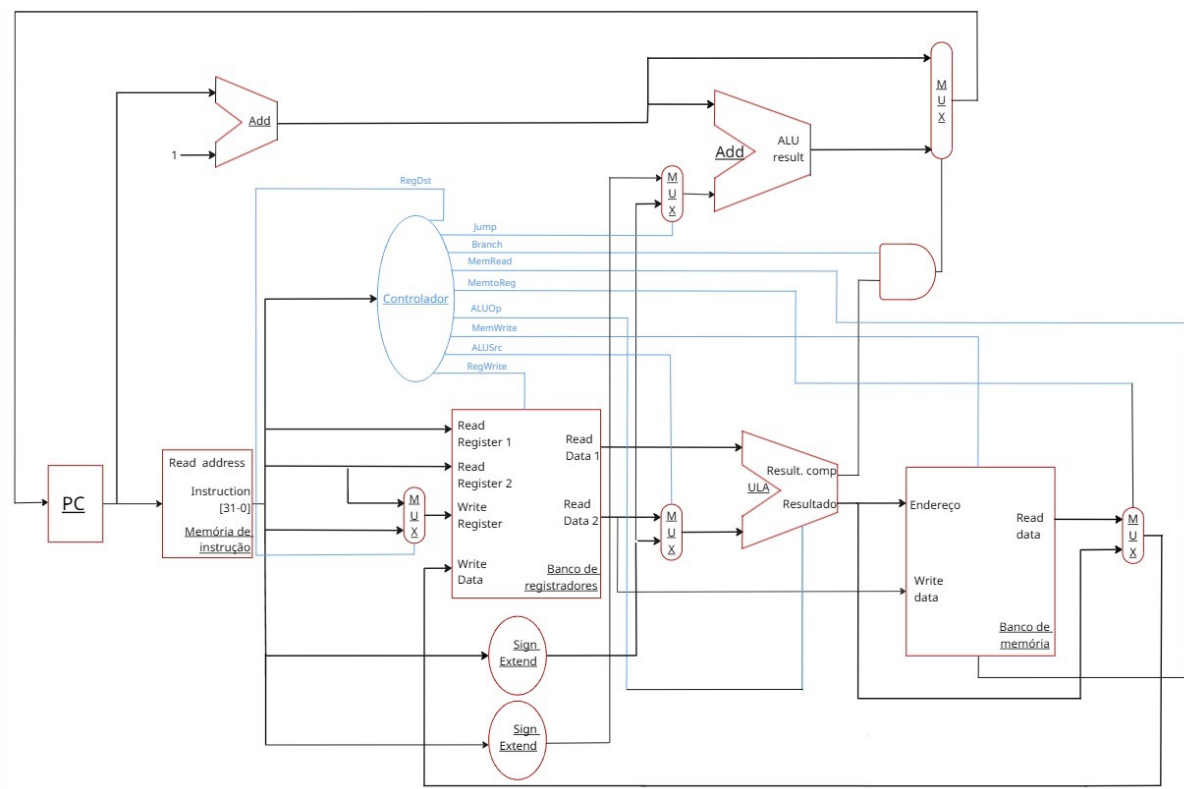
A Figura 1 apresenta a organização geral do processador em três partes principais: busca e decodificação, controle e caminho de dados. O fluxo de execução inicia-se no contador de programa, responsável por indicar o endereço da próxima instrução a ser buscada na memória de programa. A instrução obtida é decodificada e enviada à unidade de controle, que gera os sinais necessários para a operação dos demais componentes. A Figura 2 ilustra o caminho de dados do processador.

Figura 1 – Diagrama de blocos do processador



Fonte: O Autor

Figura 2 – Datapath do processador



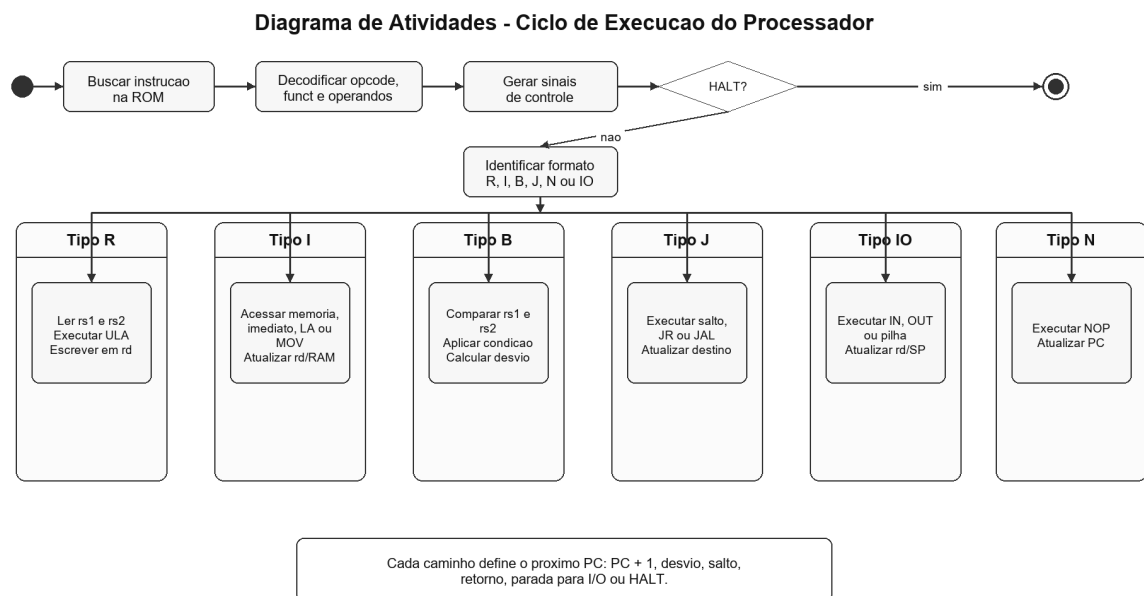
Fonte: O Autor

O processador é composto pelos seguintes blocos principais: contador de programa, memória de programa, unidade de controle, controle de desvio, banco de registradores, unidade lógica e aritmética, memória de dados, unidade de pilha e módulo de entrada e saída. Esses blocos são interligados por barramentos de dados, endereços e sinais de controle.

2.3 Ciclo de Execução do Processador

A execução de uma instrução ocorre em um ciclo sequencial composto pelas etapas de busca, decodificação, geração de sinais de controle, identificação do tipo da instrução, execução da operação e atualização do contador de programa.

Figura 3 – Diagrama de atividades do ciclo de execução do processador



Fonte: O Autor

Conforme apresentado na Figura 3, o processador inicia buscando a instrução na ROM e, em seguida, decodifica seus campos. Após a geração dos sinais de controle, a instrução é classificada conforme seu formato. Instruções do tipo R executam operações entre registradores, instruções do tipo I acessam memória ou utilizam imediato, instruções do tipo B realizam desvios condicionais, instruções do tipo J executam saltos, retornos e chamadas de função, instruções do tipo IO realizam entrada e saída, e instruções do tipo N executam operações sem operandos, como NOP ou HALT.

Para instruções diferentes de HALT, o processador executa a operação correspondente, define o próximo valor do PC e retorna ao ciclo de busca. Quando a instrução HALT é

identificada, o fluxo de execução é encerrado.

2.4 Componentes do Processador

2.4.1 Contador de Programa

O contador de programa, ou PC, armazena o endereço da instrução corrente. Em instruções sequenciais, o PC é incrementado para apontar para a próxima posição da memória de programa. Em instruções de desvio ou salto, seu valor é atualizado a partir do deslocamento calculado ou do endereço indicado pela instrução.

2.4.2 Memória de Programa

A memória de programa armazena as instruções binárias geradas pelo compilador. O arquivo executável produzido na fase final da compilação é carregado nessa memória, permitindo que o processador execute o programa a partir do endereço inicial.

2.4.3 Banco de Registradores

O banco de registradores possui 64 registradores de 32 bits. Esses registradores são utilizados para armazenar operandos, resultados temporários, endereços de memória, parâmetros de função e valores de retorno. O registrador de índice zero é tratado como registrador constante, retornando sempre o valor zero.

2.4.4 Unidade Lógica e Aritmética

A unidade lógica e aritmética executa operações aritméticas, lógicas e comparações. Ela recebe operandos provenientes do banco de registradores ou de valores imediatos e produz resultados utilizados em atribuições, expressões e decisões de desvio condicional.

2.4.5 Memória de Dados

A memória de dados armazena variáveis, vetores e valores manipulados durante a execução do programa. As instruções de carregamento e armazenamento realizam a transferência de dados entre a memória e o banco de registradores. A pilha de execução também é representada nessa memória.

2.4.6 Unidade de Controle

A decodificação da instrução é dividida entre o módulo decodificador e a unidade de controle. O decodificador separa os campos da instrução, como `opcode`, `funct`, registradores

e imediatos. A unidade de controle utiliza principalmente os campos **opcode** e **funct** para gerar os sinais responsáveis por selecionar a operação da ULA, habilitar escrita no banco de registradores, controlar acessos à memória, selecionar a origem do dado escrito em registrador, controlar desvios e habilitar operações de entrada, saída e pilha.

2.4.7 Módulo de Entrada e Saída

O módulo de entrada e saída permite a comunicação entre o processador e os periféricos da FPGA. A instrução **IN** realiza a leitura de dados externos para um registrador, enquanto a instrução **OUT** envia o conteúdo de um registrador para a saída.

2.5 Conjunto de Instruções

O conjunto de instruções do processador é composto por instruções de 32 bits. A decodificação é realizada a partir dos campos **opcode** e **funct**. O campo **opcode**, com 3 bits, identifica a classe da instrução, enquanto o campo **funct**, com 4 bits, identifica a operação específica dentro dessa classe.

As instruções foram organizadas em seis formatos: Tipo R, Tipo I, Tipo J, Tipo B, Tipo N e Tipo IO. Os campos **rs1**, **rs2** e **rd** possuem 6 bits. Os campos imediatos possuem tamanhos distintos conforme o formato da instrução, sendo utilizados para constantes, deslocamentos de memória e deslocamentos de desvio.

2.5.1 Formatos das Instruções

Tabela 1 – Formatos das instruções implementadas

Tipo	Campos da instrução	Tamanho
R	null(7) rs2(6) rs1(6) rd(6) funct(4) opcode(3)	32 bits
I	im(13) rs1(6) rd/rs2(6) funct(4) opcode(3)	32 bits
J	imm(19) rd(6) funct(4) opcode(3)	32 bits
B	im(13) rs2(6) rs1(6) funct(4) opcode(3)	32 bits
N	null(25) funct(4) opcode(3)	32 bits
IO	null(19) rd/rs1(6) funct(4) opcode(3)	32 bits

Fonte: O Autor

2.5.2 Instruções do Tipo R

As instruções do Tipo R realizam operações aritméticas e lógicas entre registradores. O campo **opcode** é fixo em 000.

Tabela 2 – Instruções do Tipo R

Instr.	Operação	funct	opcode
ADD	rd recebe $rs1 + rs2$	0000	000
SUB	rd recebe $rs1 - rs2$	0001	000
MULT	rd recebe $rs1 * rs2$	0010	000
DIV	rd recebe $rs1 / rs2$	0011	000
AND	rd recebe $rs1 \text{ AND } rs2$	0100	000
OR	rd recebe $rs1 \text{ OR } rs2$	0101	000
XOR	rd recebe $rs1 \text{ XOR } rs2$	0110	000
NOT	rd recebe NOT rs1	0111	000

Fonte: O Autor

2.5.3 Instruções do Tipo I

As instruções do Tipo I utilizam um campo imediato de 13 bits. Esse formato é utilizado em acessos à memória, operações com imediato e movimentação de dados. O campo opcode é fixo em 001. Na instrução **STORE**, o campo **rd/rs2** é utilizado como registrador fonte do valor a ser armazenado na memória.

Tabela 3 – Instruções do Tipo I

Instr.	Operação	funct	opcode
LOAD	rd recebe $\text{MEM}[rs1 + im]$	0000	001
STORE	$\text{MEM}[rs1 + im]$ recebe rs2	0001	001
ADDI	rd recebe $rs1 + im$	0010	001
SUBI	rd recebe $rs1 - im$	0011	001
ANDI	rd recebe $rs1 \text{ AND } im$	0100	001
ORI	rd recebe $rs1 \text{ OR } im$	0101	001
XORI	rd recebe $rs1 \text{ XOR } im$	0110	001
LDA	rd recebe o endereço associado ao imediato	0111	001
MOV	rd recebe rs1	1000	001

Fonte: O Autor

2.5.4 Instruções do Tipo J

As instruções do Tipo J realizam saltos incondicionais, chamadas de função e retornos indiretos. O campo opcode é fixo em 010.

Tabela 4 – Instruções do Tipo J

Instr.	Operação	funct	opcode
JUMP	Altera o PC para o endereço de destino.	0000	010
JR	Altera o PC para o endereço armazenado em registrador.	0001	010
JAL	Salva PC + 1 em rd e desvia para o destino.	0010	010

Fonte: O Autor

2.5.5 Instruções do Tipo B

As instruções do Tipo B realizam desvios condicionais. O campo `opcode` é fixo em 011. O desvio é tomado quando a comparação entre os operandos satisfaz a condição definida pelo campo `funct`.

Tabela 5 – Instruções do Tipo B

Instr.	Operação	funct	opcode
BEQ	Desvia se $rs1 = rs2$.	0000	011
BNE	Desvia se $rs1 \neq rs2$.	0001	011
BLT	Desvia se $rs1 < rs2$.	0010	011
BLE	Desvia se $rs1 \leq rs2$.	0011	011
BGT	Desvia se $rs1 > rs2$.	0100	011
BGE	Desvia se $rs1 \geq rs2$.	0101	011

Fonte: O Autor

2.5.6 Instruções do Tipo N

As instruções do Tipo N não utilizam campos de registrador nem campos imediatos. O campo `opcode` é fixo em 100.

Tabela 6 – Instruções do Tipo N

Instr.	Operação	funct	opcode
NOP	Não realiza operação.	0000	100
HALT	Interrompe a atualização do PC e finaliza a execução.	0001	100

Fonte: O Autor

2.5.7 Instruções do Tipo IO

As instruções do Tipo IO são utilizadas para entrada e saída e para manipulação da pilha. O campo `opcode` é fixo em 110. As instruções `STORE_STACK` e `LOAD_STACK` são utilizadas pelo compilador na implementação de chamadas de função, retorno, preservação de registradores e passagem de valores pela pilha.

Tabela 7 – Instruções do Tipo IO

Instr.	Operação	funct	opcode
IN	Lê dado externo para rd.	0000	110
OUT	Envia rs1 para a saída.	0001	110
STORE_STACK	Empilha o valor de rs1.	0010	110
LOAD_STACK	Desempilha valor para rd.	0011	110

Fonte: O Autor

2.6 Organização da Memória

A organização de memória utilizada pelo processador separa a memória de programa da memória de dados. A memória de programa armazena as instruções binárias geradas pelo compilador, enquanto a memória de dados armazena variáveis, vetores, parâmetros e valores manipulados durante a execução.

No projeto, a memória de programa é implementada por uma ROM inicializada a partir do arquivo `programa.txt`, gerado ao final da compilação. Esse arquivo contém uma instrução binária de 32 bits por linha, no formato esperado pelo processador. A memória de dados é acessada por instruções de carga e armazenamento, utilizando endereços calculados pela ULA.

A pilha de execução também utiliza a memória de dados. Seu controle é realizado por um registrador interno de pilha do processador, atualizado pelas instruções `STORE_STACK` e `LOAD_STACK`. Essa pilha é utilizada em chamadas de função, passagem de parâmetros, armazenamento de endereços de retorno, preservação de registradores temporários e retorno de valores. Essa organização permite a execução de chamadas aninhadas e de funções recursivas, desde que o compilador preserve os dados necessários para restaurar o contexto anterior.

2.7 Relação com o Compilador

A fase de síntese do compilador utiliza as características do processador para definir a tradução das quádruplas para assembly. Operações aritméticas e lógicas são convertidas em instruções do Tipo R ou Tipo I. Acessos a variáveis e vetores são convertidos em instruções de carga e armazenamento. Comandos condicionais e laços são convertidos em instruções de desvio e salto. Chamadas de função utilizam instruções de salto com registrador de retorno e manipulação da pilha.

Dessa forma, o conjunto de instruções, os registradores, os formatos binários e a organização da memória definem a interface entre o compilador e a arquitetura-alvo.

3 Compilador: Fase de Análise

Este capítulo descreve a fase de análise do compilador C-. Essa fase é responsável por transformar o programa-fonte em estruturas internas utilizadas pelas etapas posteriores da compilação. No projeto implementado, a análise é composta pela análise léxica, análise sintática, construção da Árvore Sintática Abstrata (AST), construção da Tabela de Símbolos e análise semântica.

A implementação utiliza o Flex para geração do analisador léxico e o Bison para geração do analisador sintático. O analisador sintático constrói a AST durante as reduções gramaticais. Em seguida, a análise semântica percorre essa árvore, registra identificadores na Tabela de Símbolos e verifica as regras semânticas implementadas.

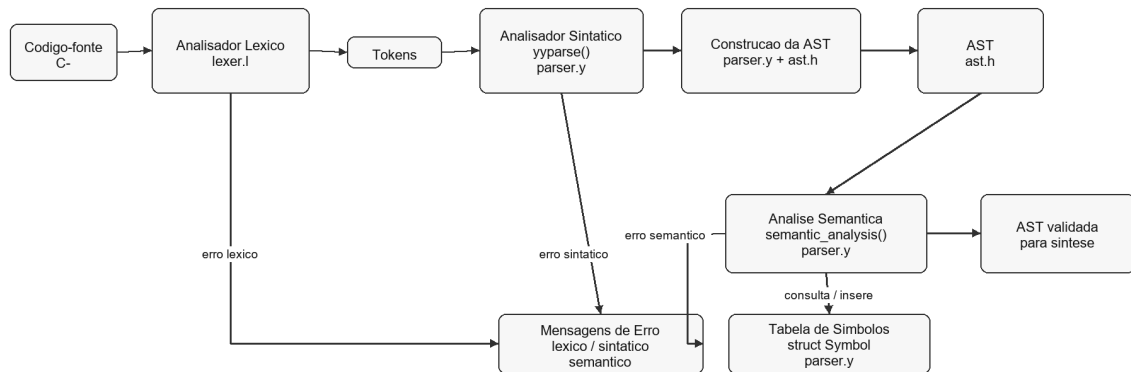
3.1 Modelagem

A modelagem da fase de análise representa o fluxo de dados entre o programa-fonte, o analisador léxico, o analisador sintático, a AST e a Tabela de Símbolos. Essa modelagem também explicita a separação entre reconhecimento de tokens, validação gramatical e verificação semântica.

3.1.1 Diagrama de Blocos (SysML)

Figura 4 – Diagrama de blocos da fase de análise do compilador

Diagrama de Blocos - Fase de Análise do Compilador



Fonte: O Autor

A Figura 4 representa os principais componentes da fase de análise do compilador. O fluxo inicia no código-fonte em C-, passa pelo analisador léxico, pelo analisador sintático e pela construção da AST. Em seguida, a AST é validada pelo analisador semântico com apoio da Tabela de Símbolos. Erros léxicos, sintáticos e semânticos são encaminhados para a geração de mensagens de erro.

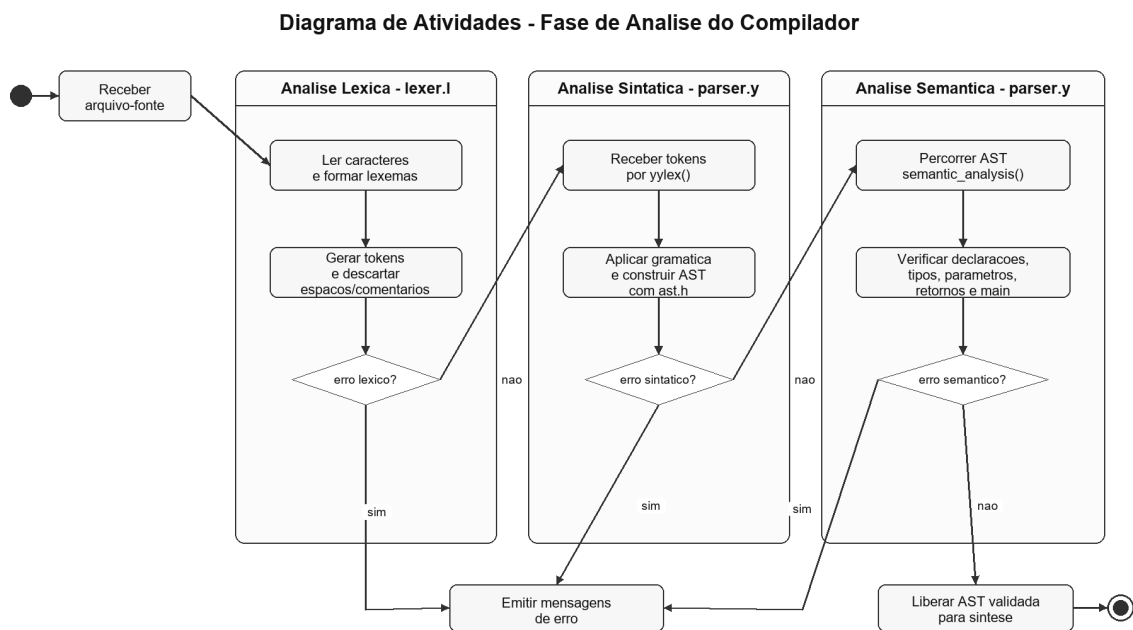
Os principais elementos representados são:

- **Programa-fonte:** arquivo de entrada escrito na linguagem C-.
- **Analisador Léxico:** componente gerado a partir do arquivo `lexer.l`, responsável por reconhecer tokens.
- **Analisador Sintático:** componente gerado a partir do arquivo `parser.y`, responsável por aplicar as regras gramaticais da linguagem.
- **Árvore Sintática Abstrata:** estrutura em memória formada por nós do tipo `Node`, definida no arquivo `ast.h`.
- **Tabela de Símbolos:** lista encadeada global utilizada pela análise semântica para registrar funções, variáveis e parâmetros.

- **Analizador Semântico:** rotina que percorre a AST e verifica as regras semânticas implementadas.

3.1.2 Diagrama de Atividades (SysML)

Figura 5 – Diagrama de atividades da fase de análise do compilador



Fonte: O Autor

A Figura 5 apresenta o fluxo de execução da fase de análise. O processo inicia com o recebimento do arquivo-fonte, seguido pela análise léxica, análise sintática e análise semântica. Caso algum erro seja identificado em uma dessas etapas, o compilador emite mensagens de erro. Quando não há erro semântico, a AST validada é liberada para a fase de síntese.

O fluxo pode ser descrito pelas seguintes etapas:

1. Leitura do arquivo-fonte contendo o programa em C-.
2. Reconhecimento de tokens pelo analisador léxico.
3. Envio dos tokens ao analisador sintático.
4. Aplicação das produções gramaticais no Bison.
5. Criação dos nós da AST durante as ações semânticas da gramática.
6. Execução da análise semântica sobre a AST.
7. Registro de símbolos e verificação das regras semânticas implementadas.

3.1.3 Estruturas Internas

A AST é definida pela estrutura `Node`, composta pelo tipo do nó, valor inteiro, identificador textual, três ponteiros para filhos, ponteiro para irmão e número da linha. Essa representação permite modelar declarações, expressões, comandos condicionais, laços, chamadas de função, retornos e acessos a vetores.

A Tabela de Símbolos é implementada como uma lista encadeada global. Cada entrada armazena nome, tipo, escopo, categoria, linha de declaração e, no caso de funções, uma lista de parâmetros. O escopo registrado diferencia principalmente o escopo global e o escopo de cada função.

3.2 Análise Léxica

A análise léxica organiza o fluxo de caracteres de entrada em tokens, etapa normalmente associada ao reconhecimento de padrões léxicos por expressões regulares (LOUDEN; SILVA, 2004). No compilador implementado, essa etapa é descrita no arquivo `lexer.l` e gerada com o Flex. As categorias reconhecidas incluem palavras-chave, identificadores, números inteiros, operadores aritméticos, operadores relacionais e delimitadores.

3.2.1 Definições Léxicas

O arquivo `lexer.l` define expressões regulares para dígitos, letras, identificadores, novas linhas e comentários de bloco. Os identificadores reconhecidos iniciam por letra e podem conter letras e dígitos.

```
1 DIGITO [0-9]
2 LETRA  [a-zA-Z]
3 ID_REGRA {LETRA}({LETRA}|{DIGITO})*
4 NOVALINHA \n
5 COMENTARIO_BLOCO "/*"([~*]|\\*+([~*/])*)~*+/"
```

Listing 3.1 – Definições léxicas utilizadas no `lexer.l`.

3.2.2 Palavras-chave

As palavras-chave da linguagem C- são reconhecidas antes da regra geral de identificadores, garantindo que lexemas reservados sejam retornados como tokens específicos.

```
1 "if"      { return IF; }
2 "int"     { return INT; }
3 "else"    { return ELSE; }
4 "return"  { return RET; }
5 "void"    { return VOID; }
6 "while"   { return WHI; }
```

Listing 3.2 – Reconhecimento de palavras-chave no `lexer.l`.

3.2.3 Operadores e Delimitadores

Os operadores relacionais, operadores aritméticos e delimitadores da linguagem são mapeados diretamente para tokens utilizados pelo parser.

```

1  "<="      { return LEQUAL; }
2  ">="      { return HEQUAL; }
3  "=="      { return IGL; }
4  "!="      { return DIF; }
5  "<"       { return MENOR; }
6  ">"       { return HIGHER; }
7  "="      { return REC; }
8  "+"      { return SOM; }
9  "-"      { return SUB; }
10 "*"      { return MUL; }
11 "/"      { return DIV; }

```

Listing 3.3 – Reconhecimento de operadores no lexer.l.

```

1  ";"      { return PEV; }
2  ","      { return VIR; }
3  "("      { return APR; }
4  ")"      { return FPR; }
5  "["      { return ACO; }
6  "]"      { return FCO; }
7  "{"      { return ACH; }
8  "}"      { return FCH; }

```

Listing 3.4 – Reconhecimento de delimitadores no lexer.l.

3.2.4 Comentários, Números e Identificadores

Comentários de bloco delimitados por `/*` e `*/` são ignorados pelo analisador léxico. Durante esse processo, o contador de linhas é atualizado quando o comentário contém quebras de linha.

```

1  {COMENTARIO_BLOCO} {
2      for(int i = 0; i < yytext[i]; i++) {
3          if(yytext[i] == '\n') {
4              yylineno++;
5          }
6      }
7  }
8  {NOVALINHA}          { yylineno++; }
9  [ \t]+               { }

```

Listing 3.5 – Tratamento de comentários e novas linhas no lexer.l.

Literais numéricos são convertidos para valores inteiros por meio da função `atoi`. Identificadores são duplicados com `strdup` e armazenados no campo semântico `sval`.

```

1  {DIGITO}+ {
2      yylval.ival = atoi(yytext);
3      return NUM;
4  }

```

```
5
6 {ID_REGRA} {
7     yylval.sval = strdup(yytext);
8     return ID;
9 }
```

Listing 3.6 – Reconhecimento de números e identificadores no lexer.l.

3.2.5 Tratamento de Erros Léxicos

Caracteres que não correspondem a nenhuma regra léxica são tratados pela regra curinga. O compilador imprime uma mensagem contendo o caractere reconhecido e a linha correspondente.

```
1 . {
2     printf("ERRO LEXICO: %s LINHA: %d\n", yytext, yylineno);
3 }
```

Listing 3.7 – Regra de erro léxico no lexer.l.

No código implementado, o erro léxico é reportado por mensagem na saída padrão. A regra não incrementa um contador global de erros léxicos.

3.3 Análise Sintática

A análise sintática processa o fluxo de tokens fornecido pelo analisador léxico e verifica se a sequência de tokens pertence à gramática da linguagem C-. Essa etapa é implementada no arquivo `parser.y`, utilizando Bison ([Free Software Foundation, 2024](#)).

Além de validar a estrutura gramatical, o parser constrói a AST durante as ações semânticas associadas às produções. A raiz da árvore é armazenada na variável global `ast_root`.

3.3.1 Declaração de Tokens e Tipos

O parser utiliza a diretiva `%union` para definir os tipos de valores semânticos manipulados pela pilha do Bison. No compilador implementado, esses valores podem ser inteiros, strings ou ponteiros para nós da AST.

```
1 %union {
2     int ival;
3     char* sval;
4     struct Node* nptr;
5 }
```

Listing 3.8 – Declaração da união semântica no parser.y.

Os tokens `NUM` e `ID` possuem valores semânticos associados. Os demais tokens representam palavras-chave, operadores e delimitadores reconhecidos pelo lexer.


```

1 %token <ival> NUM
2 %token <sval> ID
3 %token IF ELSE INT RET VOI WHI
4 %token SOM SUB MUL DIV
5 %token MENOR LEQUAL HIGHER HEQUAL IGL DIF
6 %token REC
7 %token PEV VIR
8 %token APR FPR
9 %token ACO FCO
10 %token ACH FCH

```

Listing 3.9 – Trecho da declaração de tokens no parser.y.

3.3.2 Construção da AST

A construção da AST ocorre nas ações semânticas das produções gramaticais. Cada produção cria nós compatíveis com o tipo sintático reconhecido, utilizando as funções

`create_node`, `create_leaf_val` e `create_leaf_id`.

```

1 var_declaracao:
2     tipo_especificador ID PEV {
3         $$ = create_node(NODE_VAR_DECLARACAO, $1, create_leaf_id(NODE_ID, $2), NULL);
4     }
5     | tipo_especificador ID ACO NUM FCO PEV {
6         $$ = create_node(NODE_VAR_DECLARACAO_ARRAY, $1, create_leaf_id(NODE_ID, $2),
7             create_leaf_val(NODE_NUM, $4));
8     }
9     ;

```

Listing 3.10 – Produções responsáveis por declarações de variáveis no parser.y.

Chamadas de função, acessos a vetores, expressões aritméticas e comandos de controle também são representados por nós específicos da AST.

```

1 var:
2     ID { $$ = create_leaf_id(NODE_ID, $1); }
3     | ID ACO expressao FCO {
4         $$ = create_node(NODE_VAR_ARRAY, create_leaf_id(NODE_ID, $1), $3, NULL);
5     }
6     ;
7
8 ativacao:
9     ID APR args FPR {
10         $$ = create_node(NODE_ATIVACAO, create_leaf_id(NODE_ID, $1), $3, NULL);
11     }
12     ;
13
14 fator:
15     APR expressao FPR { $$ = $2; }
16     | var { $$ = $1; }
17     | ativacao { $$ = $1; }
18     | NUM { $$ = create_leaf_val(NODE_NUM, $1); }
19     ;

```

Listing 3.11 – Produções para variáveis, chamadas e fatores no parser.y.

3.3.3 Precedência e Ambiguidade

O parser define precedência para operadores relacionais, aritméticos e atribuição. O conflito associado ao *else* pendente é tratado com as diretivas `LOWER_THAN_ELSE` e `ELSE`.

```

1 %nonassoc LEQUAL HEQUAL MENOR HIGHER IGL DIF
2 %left SOM SUB
3 %left MUL DIV
4 %right REC
5 %nonassoc LOWER_THAN_ELSE
6 %nonassoc ELSE

```

Listing 3.12 – Diretivas de precedência utilizadas no parser.y.

3.3.4 Tratamento de Erros Sintáticos

O arquivo `parser.y` possui produções com o símbolo especial `error` para recuperação em declarações e comandos. Quando o Bison detecta erro sintático, a função `yyerror` incrementa o contador `syntax_error_count` e imprime a linha correspondente.

```

1 declaracao:
2     var_declaracao { $$ = $1; }
3     | fun_declaracao { $$ = $1; }
4     | error PEV { yyerrok; $$ = NULL; }
5     ;
6
7 statement:
8     expressao_decl { $$ = $1; }
9     | composto_decl { $$ = $1; }
10    | selecao_decl { $$ = $1; }
11    | iteracao_decl { $$ = $1; }
12    | retorno_decl { $$ = $1; }
13    | error PEV { yyerrok; $$ = NULL; }
14    ;

```

Listing 3.13 – Tratamento de erro sintático no parser.y.

3.3.5 Exemplo de AST

O código a seguir representa um programa C- válido utilizado para ilustrar a estrutura de nós da AST.

```

1 void main(void) {
2     int x;
3     int y;
4     x = 3;
5     y = x + 1;
6     return;
7 }

```

Listing 3.14 – Código C- utilizado como exemplo de AST.

A Figura 6 apresenta uma representação simplificada da AST correspondente. Essa representação omite alguns detalhes de ligação interna, como o encadeamento por `sibling` e o armazenamento da lista de parâmetros no nó identificador da função.

```
PROGRAMA
  FUN_DECLARACAO
    TIPO: VOID
    ID (main)
    TIPO: VOID
  COMPOSTO_DECL
    VAR_DECLARACAO
      TIPO: INT
      ID (x)
    VAR_DECLARACAO
      TIPO: INT
      ID (y)
    ATRIBUICAO (=)
      ID (x)
      NUM (3)
    ATRIBUICAO (=)
      ID (y)
      OPERADOR (+)
      ID (x)
      NUM (1)
    RETORNO_DECL
```

Figura 6 – Representação simplificada da AST gerada para o código-fonte de exemplo.

3.4 Tabela de Símbolos

A Tabela de Símbolos é utilizada pela análise semântica para registrar identificadores declarados no programa e consultar informações de tipo, escopo e categoria. No compilador implementado, a tabela principal é uma lista encadeada global apontada pela variável `symbol_table`.

3.4.1 Estrutura de Dados

Cada entrada da tabela é representada pela estrutura `Symbol`. O campo `params` é utilizado em símbolos de função para armazenar a lista de parâmetros esperados.

```
1 typedef struct Symbol {
2     char* name;
3     char* type;
4     char* scope;
5     char* kind;
6     int lineno;
```

```
7     struct Symbol* next;
8     struct Symbol* params;
9 } Symbol;
10
11 Symbol* symbol_table = NULL;
```

Listing 3.15 – Estrutura da Tabela de Símbolos em parser.y.

3.4.2 Inserção e Busca

A inserção de símbolos ocorre no início da lista encadeada. A busca por variáveis é feita primeiro no escopo corrente e, caso não haja correspondência, no escopo global.

```
1 void insert_symbol(char* name, char* type, char* scope, char* kind, int lineno) {
2     Symbol* s = create_symbol(name, type, scope, kind, lineno);
3     s->next = symbol_table;
4     symbol_table = s;
5 }
```

Listing 3.16 – Inserção de símbolos em parser.y.

```
1 Symbol* lookup_symbol(char* name, char* scope) {
2     Symbol* s = symbol_table;
3     while (s != NULL) {
4         if (strcmp(s->name, name) == 0 && strcmp(s->scope, scope) == 0) {
5             return s;
6         }
7         s = s->next;
8     }
9
10    s = symbol_table;
11    while (s != NULL) {
12        if (strcmp(s->name, name) == 0 && strcmp(s->scope, "global") == 0) {
13            return s;
14        }
15        s = s->next;
16    }
17    return NULL;
18 }
```

Listing 3.17 – Busca de símbolo por escopo em parser.y.

3.4.3 Símbolos Pré-definidos

Antes da análise do programa, o compilador insere as funções pré-definidas `input` e `output`. A função `input` possui retorno inteiro e não recebe parâmetros. A função `output` possui retorno `void` e recebe um parâmetro inteiro.

```
1 insert_symbol("input", "int", "global", "fun", 0);
2
3 insert_symbol("output", "void", "global", "fun", 0);
4 Symbol* out_sym = lookup_function("output");
5 Symbol* param = create_symbol("x", "int", "output", "param", 0);
6 out_sym->params = param;
```

Listing 3.18 – Inserção das funções pré-definidas na análise semântica.

3.5 Análise Semântica

A análise semântica percorre a AST e verifica regras que dependem de contexto. No compilador implementado, essa etapa é iniciada pela função `semantic_analysis`, que insere as funções pré-definidas, chama `analyze_node` sobre a raiz da AST e, ao final, verifica a existência e a assinatura da função `main`.

As principais verificações implementadas são:

- declaração inválida de variável do tipo `void`;
- redeclaração de identificador no mesmo escopo registrado;
- conflito entre declaração de variável e função já declarada;
- declaração duplicada de função;
- inserção e validação de parâmetros de função;
- chamada de função não declarada;
- chamada de função com número incorreto de argumentos;
- chamada de função com tipo de argumento incompatível;
- uso de variável não declarada;
- atribuição direta a vetor sem índice;
- atribuição de expressão `void` a variável inteira;
- retorno com valor em função `void`;
- função `int` sem retorno com expressão;
- ausência da função `main`;
- função `main` com tipo diferente de `void`;
- função `main` com parâmetros.

3.5.1 Verificação de Declarações

Durante a análise de declarações de variáveis, o compilador verifica o tipo declarado, a existência de declaração anterior no mesmo escopo registrado e conflito com funções declaradas.

```
1 if (strcmp(type, "void") == 0) {
2     semantic_error("declaracao invalida de variavel", name, node->lineno);
3 }
4 else if (is_declared_in_scope(name, current_scope)) {
5     semantic_error("declaracao invalida de variavel", name, node->lineno);
6 }
7 else if (lookup_function(name)) {
8     semantic_error("declaracao invalida", name, node->lineno);
9 } else {
10     insert_symbol(name, type, current_scope,
11         (node->type == NODE_VAR_DECLARACAO_ARRAY) ? "array" : "var",
12         node->lineno);
13 }
```

Listing 3.19 – Verificação de declarações de variáveis na análise semântica.

3.5.2 Verificação de Funções

Na análise de funções, o compilador registra a função no escopo global, insere seus parâmetros no escopo da função e verifica se funções do tipo `int` possuem ao menos um comando `return` com expressão.

```
1 if (strcmp(type, "int") == 0) {
2     if (!function_has_return(node->child3)) {
3         semantic_error("funcao int sem retorno", name, node->lineno);
4     }
5 }
```

Listing 3.20 – Verificação de função do tipo `int` sem retorno.

A verificação implementada identifica a presença de algum retorno com expressão dentro do corpo da função. Ela não realiza uma análise completa de todos os caminhos possíveis de execução.

3.5.3 Verificação de Chamadas de Função

Para chamadas de função, o analisador verifica se a função foi declarada, compara a quantidade de argumentos fornecidos com a quantidade de parâmetros registrados e valida os tipos quando ambos são conhecidos.

```
1 if (arg_count != param_count) {
2     semantic_error("chamada invalida", "numero de parametros invalido", node->lineno);
3 }
```

Listing 3.21 – Trecho da verificação de quantidade de argumentos em chamadas de função.

3.5.4 Verificação de Variáveis e Vetores

O uso de identificadores é validado por consulta à Tabela de Símbolos. Em atribuições, o compilador também impede a atribuição direta a um vetor sem índice.

```
1 if (sym && strcmp(sym->kind, "array") == 0) {  
2     semantic_error("atribuicao invalida a array", var_node->sval, node->lineno);  
3 }
```

Listing 3.22 – Verificação de atribuição direta a vetor.

3.5.5 Verificação da Função Main

Ao final da análise semântica, o compilador verifica se a função `main` foi declarada, se seu tipo é `void` e se ela não possui parâmetros.

```
1 Symbol* main_sym = lookup_function("main");  
2 if (main_sym == NULL) {  
3     semantic_error("funcao main() nao declarada", NULL, 0);  
4 } else {  
5     if (strcmp(main_sym->type, "void") != 0) {  
6         semantic_error("funcao main deve ser do tipo void", "main", main_sym->lineno);  
7     }  
8     if (main_sym->params != NULL) {  
9         semantic_error("funcao main nao deve ter parametros", "main", main_sym->lineno);  
10    }  
11 }
```

Listing 3.23 – Verificação da função `main` na análise semântica.

O compilador atual não verifica se a função `main` é a última declaração do arquivo-fonte. A restrição implementada considera apenas existência, tipo de retorno e ausência de parâmetros.

4 Compilador: Fase de Síntese

A fase de síntese é executada após a análise sintática e semântica do programa-fonte. No compilador implementado, essa fase recebe como entrada principal a Árvore Sintática Abstrata (AST) e percorre seus nós para gerar código intermediário, código assembly e código binário para o processador-alvo.

A implementação da fase de síntese está concentrada no arquivo `gerador_quaduplas.c`. A função `iniciar_compilador` inicializa as estruturas internas do gerador, percorre a AST por meio da função `processa_arvore` e, ao final, chama `gerar_saidas_finais` para produzir os arquivos de saída.

A saída da fase de síntese é composta por três arquivos: `saida.quad`, contendo as quádruplas geradas; `saida.asm`, contendo o código assembly correspondente; e `programa.txt`, contendo as instruções binárias de 32 bits utilizadas para inicializar a memória de instruções do processador.

Embora a fase de análise possua uma Tabela de Símbolos própria para validação semântica, o gerador de código utiliza uma estrutura interna específica para o mapeamento de variáveis em memória. Essa estrutura é composta por entradas do tipo `VarMap`, que armazenam nome, escopo, endereço base e informações sobre vetores.

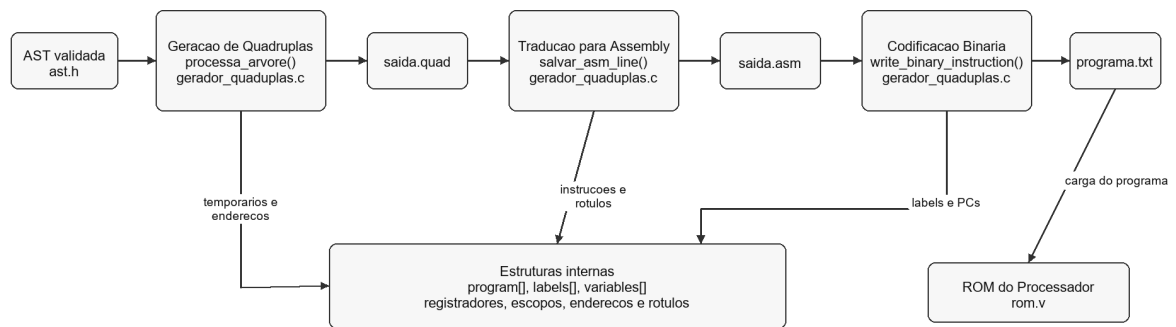
4.1 Modelagem

A modelagem da fase de síntese representa o fluxo de transformação da AST em três representações sucessivas: quádruplas, assembly e código binário. Essa etapa também inclui o mapeamento de variáveis em memória, a geração de rótulos, o gerenciamento de registradores temporários e o tratamento da pilha de execução.

4.1.1 Diagrama de Blocos (SysML)

Figura 7 – Diagrama de blocos da fase de síntese do compilador

Diagrama de Blocos - Fase de Síntese do Compilador



Fonte: O Autor

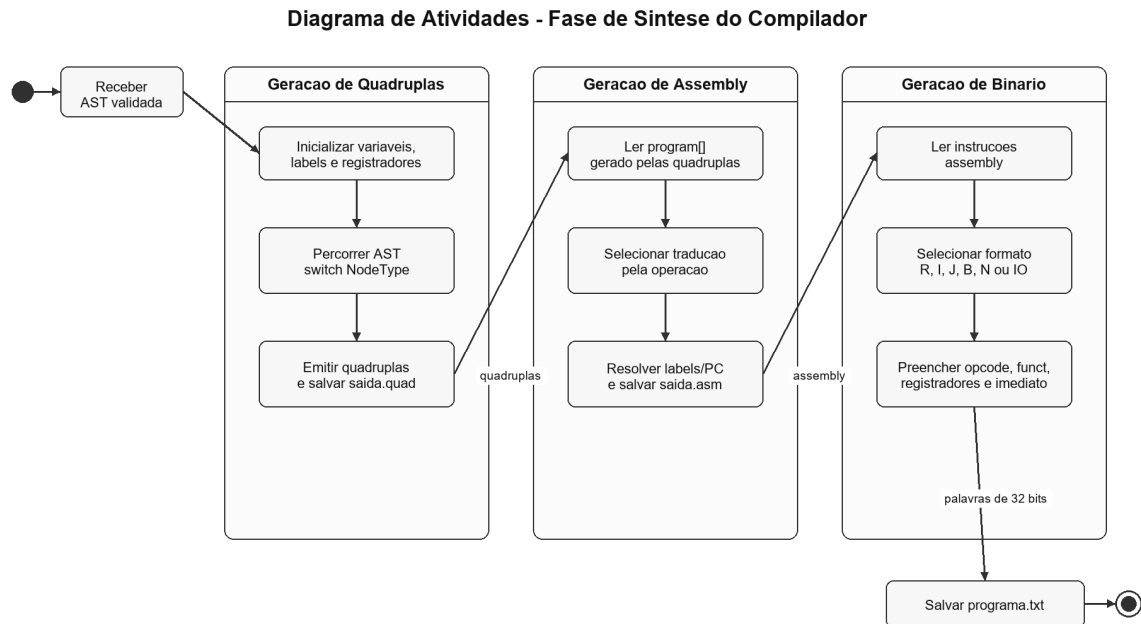
A Figura 7 apresenta a transformação da AST validada nas representações finais utilizadas pelo processador. A fase inicia com a geração de quádruplas, passa pela geração de código assembly e termina com a codificação binária das instruções. O arquivo `programa.txt` produzido ao final é carregado na ROM do processador.

Os principais elementos representados são:

- **AST:** estrutura de entrada produzida pela fase de análise.
- **Gerador de Quádruplas:** percorre a AST e emite operações intermediárias.
- **Mapeador de Memória:** registra variáveis, vetores e parâmetros na estrutura `VarMap`.
- **Gerador de Assembly:** traduz quádruplas e diretivas internas para instruções assembly.
- **Resolvedor de Rótulos:** associa labels e funções a posições reais do código.
- **Codificador Binário:** monta as instruções de 32 bits conforme a ISA do processador.
- **Arquivos de Saída:** `saida.quad`, `saida.asm` e `programa.txt`.

4.1.2 Diagrama de Atividades (SysML)

Figura 8 – Diagrama de atividades da fase de síntese do compilador



Fonte: O Autor

A Figura 8 detalha as atividades executadas na fase de síntese. A AST validada é inicialmente percorrida para geração das quádruplas. Em seguida, essas quádruplas são traduzidas para assembly, com resolução de rótulos e expansão de operações quando necessário. Por fim, as instruções assembly são convertidas para palavras binárias de 32 bits e gravadas no arquivo `programa.txt`.

O fluxo da fase de síntese pode ser descrito pelas seguintes etapas:

1. Inicialização dos arquivos de saída e das estruturas internas do gerador.
2. Percurso da AST a partir do nó raiz.
3. Registro de variáveis, vetores e parâmetros no mapeamento interno de memória.
4. Geração de quádruplas para expressões, comandos, chamadas de função, retornos, entrada e saída.
5. Conversão das quádruplas e diretivas internas para uma lista de instruções assembly.
6. Primeira passagem sobre a lista de instruções para mapear rótulos e calcular posições reais do contador de programa.
7. Segunda passagem para escrever o assembly e codificar as instruções em binário.

8. Gravação dos arquivos `saida.quad`, `saida.asm` e `programa.txt`.

4.1.3 Estruturas Auxiliares

A fase de síntese utiliza estruturas auxiliares para armazenar instruções assembly, rótulos e variáveis mapeadas em memória.

```

1 typedef struct {
2     char op[16];
3     char arg1[64];
4     char arg2[64];
5     char arg3[64];
6     int is_label;
7     char label_name[64];
8 } AsmInstruction;
9
10 typedef struct {
11     char name[64];
12     int pc;
13 } LabelMap;
14
15 typedef struct {
16     char name[64];
17     int addr;
18     char scope[64];
19     int is_array_param;
20     int is_array;
21 } VarMap;

```

Listing 4.1 – Estruturas auxiliares da fase de síntese.

4.2 Geração do Código Intermediário

Representações intermediárias são utilizadas para separar a análise do programa-fonte da geração final de código para a arquitetura-alvo (AHO et al., 2007). A função `processa_arvore` identifica o tipo de cada nó e emite operações intermediárias por meio da função `salvar_arquivo`. As quádruplas são gravadas no arquivo `saida.quad` pela função `salvar_quad`.

```

1 void salvar_quad(char* OP, char* A1, char* A2, char* RES) {
2     FILE *f = fopen("saida.quad", "a");
3     if (f) {
4         fprintf(f, "(%s, %s, %s, %s)\n", OP, A1, A2, RES);
5         fclose(f);
6     }
7 }

```

Listing 4.2 – Gravação de quádruplas no arquivo `saida.quad`.

4.2.1 Formato das Quádruplas

As quádruplas são representadas no formato `(OP, ARG1, ARG2, RES)`. O campo `OP` identifica a operação. Os campos `ARG1` e `ARG2` representam operandos, variáveis, registradores ou

rótulos. O campo `RES` representa o destino da operação ou informação auxiliar, dependendo da instrução.

4.2.2 Tipos de Quádruplas Definidos

A Tabela 8 apresenta as principais quádruplas emitidas pelo compilador.

Tabela 8 – Principais quádruplas geradas na fase de síntese

Quádrupla	Função
ADD, SUB, MULT, DIV	Operações aritméticas entre registradores temporários.
LOAD	Carregamento de variável ou posição de vetor para registrador.
STORE	Armazenamento de valor em variável ou posição de vetor.
LOADI	Carregamento de constante imediata em registrador temporário.
JUMP	Desvio incondicional para rótulo ou função.
LAB	Marcação de rótulo no código intermediário.
BEQ, BNE, BLT, BLE, BGT, BGE	Desvios condicionais gerados para comandos condicionais e laços.
FUN	Marcação do início de uma função.
ARG	Recebimento de argumento no início de uma função.
PARAM	Empilhamento de argumento antes de uma chamada de função.
CALL	Chamada de função e recuperação do valor de retorno.
RET	Retorno de função.
END	Marcação de fim de função.
SAVE_CONTEXT	Diretiva intermediária para preservação de contexto antes de chamadas.
IN, OUT	Operações de entrada e saída.
HALT	Finalização do programa.

Fonte: O Autor

A diretiva `ALLOC` é utilizada internamente para registrar variáveis e vetores no mapeamento de memória. No código atual, essa diretiva não é gravada no arquivo `saida.quad` e não gera instrução assembly diretamente.

4.3 Geração do Código Assembly

A geração do código assembly é realizada a partir das operações intermediárias emitidas durante o percurso da AST. No código implementado, essa tradução é centralizada na função `salvar_arquivo`, que recebe os campos `OP`, `ARG1`, `ARG2` e `RES` da quádrupla e decide qual sequência de instruções assembly deve ser registrada na lista interna `program`.

Algumas quádruplas possuem tradução direta para uma única instrução assembly. Outras são pseudo-instruções da representação intermediária e são expandidas para uma sequência de instruções da arquitetura-alvo. Essa separação permite manter o código intermediário mais próximo da estrutura lógica do programa e transferir para a geração de

assembly os detalhes de pilha, chamadas de função, passagem de parâmetros e restauração de contexto.

As Tabelas 9 e 10 apresentam o mapeamento das principais quádruplas utilizadas pelo compilador para o código assembly.

Tabela 9 – Mapeamento de quádruplas aritméticas, memória e controle

Categoria	Quádrupla	Tradução para assembly
Aritmética	(ADD, a, b, r)	ADD r, a, b
Aritmética	(SUB, a, b, r)	SUB r, a, b
Aritmética	(MULT, a, b, r)	MULT r, a, b
Aritmética	(DIV, a, b, r)	DIV r, a, b
Constante	(LOADI, n, -, r)	ADDI r, \$zero, n
Memória	(LOAD, x, -, r)	LOAD r, x
Memória	(STORE, v, x, -)	STORE -, v, x
Vetores	(LOAD, vet, i, r)	Sequência de cálculo de endereço e leitura de vet[i].
Vetores	(STORE, v, vet, i)	Sequência de cálculo de endereço e escrita em vet[i].
Controle	(LAB, L, -, -)	L:
Controle	(JUMP, L, -, -)	JUMP L
Controle	(BEQ, a, b, L)	BEQ a, b, L
Controle	(BNE, a, b, L)	BNE a, b, L
Controle	(BLT, a, b, L)	BLT a, b, L
Controle	(BLE, a, b, L)	BLE a, b, L
Controle	(BGT, a, b, L)	BGT a, b, L
Controle	(BGE, a, b, L)	BGE a, b, L

Fonte: O Autor

Tabela 10 – Mapeamento de quádruplas de função, contexto e entrada/saída

Categoria	Quádrupla	Tradução para assembly
Função	(FUN, tipo, f, -)	Geração do rótulo f:.
Função	(ARG, x, -, -)	LOAD_STACK \$at seguido de STORE \$at, x, 0.
Função	(PARAM, v, -, -)	STORE_STACK v.
Função	(CALL, f, n, r)	JAL \$ra, f, recuperação do retorno e restauração de contexto.
Função	(RET, v, -, -)	LOAD_STACK \$ra, STORE_STACK v e JR \$ra.
Função	(RET, -, -, -)	LOAD_STACK \$ra e JR \$ra.
Função	(END, f, -, -)	Finalização lógica da função; em funções diferentes de main, pode emitir retorno por JR.
Contexto	(SAVE_CONTEXT, escopo, n, -)	Preserva variáveis locais e registradores temporários antes de chamadas.
Entrada/Saída	(IN, -, -, r)	IN r.
Entrada/Saída	(OUT, v, -, -)	OUT v.
Finalização	(HALT, -, -, -)	HALT.

Fonte: O Autor

A tradução de comandos condicionais e laços utiliza diretamente instruções de desvio. Quando a condição é uma expressão relacional, o compilador gera o branch invertido

para saltar ao rótulo de saída quando a condição é falsa. Por exemplo, uma condição `a < b` em um `if` pode gerar a instrução `BGE a, b, L`, desviando para o rótulo `L` quando a condição original não é satisfeita.

A lista interna de instruções assembly não é gravada diretamente no arquivo final no momento da emissão. Ela é armazenada em memória para permitir uma etapa posterior de resolução de rótulos. Essa resolução é necessária porque rótulos de funções, rótulos de controle e acessos a vetores dependem do valor real do contador de programa após a expansão das instruções.

4.4 Geração do Código Executável

A geração do código executável corresponde à codificação das instruções assembly em palavras binárias de 32 bits. O resultado é armazenado no arquivo `programa.txt`, no qual cada linha representa uma instrução em formato binário. Esse arquivo é carregado pela memória de programa do processador.

A função `gerar_saidas_finais` realiza essa etapa em duas passagens. A primeira passagem não gera código de saída; ela calcula o valor real do contador de programa associado a cada rótulo. Esse cálculo é necessário porque algumas instruções simbólicas, principalmente acessos a vetores, são expandidas em múltiplas instruções binárias. Assim, o endereço de um rótulo não pode ser obtido apenas pela posição da instrução na lista interna.

A segunda passagem escreve o arquivo `saida.asm` e codifica as instruções no arquivo `programa.txt`. Nessa etapa, saltos e desvios utilizam os endereços calculados na primeira passagem para gerar os deslocamentos relativos corretos.

A Tabela 11 apresenta a organização dos campos binários utilizados pelo codificador.

Tabela 11 – Formatos de codificação das instruções

Tipo	Instruções	Formato dos campos	Opcode
R	ADD, SUB, MULT, DIV, AND, OR, XOR	null(7) rs2(6) rs1(6) rd(6) funct(4) opcode(3)	000
I	LOAD, STORE, ADDI, SUBI, ANDI, ORI, XORI	im(13) rs1(6) rd/rs2(6) funct(4) opcode(3)	001
J	JUMP, JAL, JR	offset(19) rd(6) funct(4) opcode(3)	010
B	BEQ, BNE, BLT, BLE, BGT, BGE	offset(13) rs2(6) rs1(6) funct(4) opcode(3)	011
N	HALT	null(25) funct(4) opcode(3)	100
IO	IN, OUT, STORE_STACK, LOAD_STACK	null(19) rd/rs1(6) funct(4) opcode(3)	110

Fonte: O Autor

Como exemplo, a instrução assembly `ADD $t0, $t1, $t2` é codificada como uma instrução do Tipo R. O registrador de destino `$t0` ocupa o campo `rd`, enquanto `$t1` e `$t2` ocupam os campos `rs1` e `rs2`. O campo `funct` recebe o valor 0000, correspondente à operação `ADD`, e o campo `opcode` recebe 000.

Para instruções de desvio, o campo imediato não armazena o endereço textual do rótulo. O compilador calcula o deslocamento relativo pela expressão `target - current_pc - 1`, onde `target` é o contador de programa associado ao rótulo de destino. Esse deslocamento é então colocado no campo `offset` da instrução binária.

4.5 Gerenciamento de Memória

O gerenciamento de memória da fase de síntese é feito por uma tabela interna de variáveis. Cada variável recebe um endereço base na memória de dados. Variáveis simples ocupam uma palavra de 32 bits. Vetores ocupam uma sequência de palavras, com tamanho determinado pela declaração.

```
1 strcpy(variables[var_count].name, A1);
2 strcpy(variables[var_count].scope, scope);
3 variables[var_count].addr = next_ram_addr;
4 variables[var_count].is_array_param = 0;
5 variables[var_count].is_array = is_arr;
6 next_ram_addr += size * 4;
7 var_count++;
```

Listing 4.3 – Registro de variável no mapeamento interno de memória.

A busca de endereços considera primeiro o escopo atual e, caso não encontre a variável, consulta o escopo global. Esse comportamento permite diferenciar variáveis globais e variáveis associadas a funções.

4.5.1 Gerenciamento de Registradores

O compilador utiliza registradores temporários no formato `$t0`, `$t1`, `$t2` e assim por diante. A função `alocar_reg` percorre um vetor de controle e retorna o primeiro registrador temporário livre. Após o uso, a função `liberar_reg` marca o registrador como disponível.

```
1 for(i = 0; i < 60; i++) {
2     if (regs_vetor[i] == 0) {
3         regs_vetor[i] = 1;
4         char* t = (char*) malloc(16);
5         sprintf(t, "%t%d", i);
6         return t;
7     }
8 }
```

Listing 4.4 – Alocação de registradores temporários.

O mapeamento entre nomes simbólicos de registradores e índices físicos é feito pela função `get_reg_num`. O registrador `$zero` é mapeado para o índice 0, `$ra` para o índice 1, `$sp` para o índice 2, `$at` para o índice 3 e os registradores `$tN` para os índices a partir de 4.

4.5.2 Pilha de Execução e Chamadas de Função

A pilha é utilizada para passagem de parâmetros, armazenamento de endereço de retorno, retorno de valores e preservação de contexto. Antes de chamadas de função, a diretiva `SAVE_CONTEXT` provoca o salvamento de variáveis locais do escopo corrente e dos registradores temporários em uso.

A passagem de parâmetros é feita pela emissão de `PARAM`, traduzida para `STORE_STACK`. No início da função chamada, as diretivas `ARG` são traduzidas para `LOAD_STACK` seguido de armazenamento do argumento na posição de memória associada ao parâmetro.

Chamadas de função são traduzidas para `JAL`. Retornos são traduzidos para restauração do registrador `$ra`, empilhamento do valor de retorno quando aplicável e instrução `JR`.

4.5.3 Vetores

Vetores declarados no programa são alocados em posições consecutivas da memória de dados. Como cada palavra possui 32 bits, o acesso ao elemento de índice `i` exige o cálculo do deslocamento $4 * i$ em relação ao endereço base.

Para vetores locais ou globais, o compilador calcula o deslocamento por somas sucessivas, adiciona o endereço base e executa a instrução de carga ou armazenamento. Para vetores recebidos como parâmetro, o valor armazenado na posição do parâmetro é tratado como endereço base do vetor. Dessa forma, o acesso a vetores passados por referência é expandido em uma sequência maior de instruções.

No código gerado, acessos a vetores podem ser expandidos em múltiplas instruções binárias. Por isso, a primeira passagem da função `gerar_saidas_finais` considera essa expansão ao calcular os valores reais dos rótulos.

5 Exemplos de Uso do Compilador

Neste capítulo, são apresentados os resultados práticos da compilação de três programas escritos em C-: o algoritmo de Euclides para cálculo do máximo divisor comum, o fatorial recursivo e o algoritmo de ordenação por seleção. Os exemplos demonstram o fluxo completo de compilação, incluindo a geração do código intermediário em quádruplas, a tradução para assembly e a codificação das instruções em palavras binárias de 32 bits.

Em cada caso, o código-fonte é apresentado antes da tabela de correspondência. Em seguida, a tabela completa relaciona as quádruplas geradas, as instruções assembly correspondentes e o código binário produzido. As linhas de rótulo no assembly não possuem palavra binária associada, pois representam apenas endereços simbólicos resolvidos durante a geração do código executável.

5.1 Algoritmo de Euclides: Máximo Divisor Comum

O primeiro caso de teste é o algoritmo de Euclides, implementado com chamada recursiva. Esse exemplo exercita comandos condicionais, operações aritméticas, divisão inteira, chamada de função com dois argumentos, retorno de valor e comandos de entrada e saída.

5.1.1 Código-fonte

```
1  /* Um programa para calcular o mdc
2     segundo o algoritmo de Euclides. */
3
4  int gcd(int u, int v)
5  {
6      if (v == 0)
7          return u;
8      else
9          return gcd(v, u - u / v * v);
10 }
11
12 void main(void)
13 {
14     int x;
15     int y;
16
17     x = input();
18     y = input();
19
20     output(gcd(x, y));
21 }
```

Listing 5.1 – Código-fonte do algoritmo de Euclides para cálculo do MDC.

5.1.2 Tabela de correspondência

Tabela 12 – Tabela completa de correspondência do Algoritmo de Euclides (MDC)

Quádruplas	Assembly (PC)	Código Binário (PC)
1: (JUMP, main, -, -)	0: JUMP main	0: 0000000000000011 1000000000000010
2: (FUN, int, gcd, -)	gcd:	
3: (ARG, u, -, -)	1: LOAD_STACK \$at	1: 0000000000000000 0000000110011110
4: (ARG, v, -, -)	2: STORE \$at, u, 0	2: 0000000000000000 0000000110001001
5: (LOAD, v, -, \$t0)	3: LOAD_STACK \$at	3: 0000000000000000 0000000110011110
6: (LOADI, 0, -, \$t1)	4: STORE \$at, v, 0	4: 0000000000100000 0000000110001001
7: (BNE, \$t0, \$t1, L0)	5: STORE_STACK \$ra	5: 0000000000000000 0000000010010110
8: (LOAD, u, -, \$t0)	6: LOAD \$t0, v	6: 0000000000100000 0000001000000001
9: (RET, \$t0, -, -)	7: ADDI \$t1, \$zero, 0	7: 0000000000000000 0000001010010001
10: (JUMP, L1, -, -)	8: BNE \$t0, \$t1, L0	8: 0000000000101000 1010001000001011
11: (LAB, L0, -, -)	9: LOAD \$t0, u	9: 0000000000000000 0000001000000001
12: (LOAD, u, -, \$t0)	10: LOAD_STACK \$ra	10: 0000000000000000 0000000010011110
13: (LOAD, u, -, \$t1)	11: STORE_STACK \$t0	11: 0000000000000000 0000001000010110
14: (LOAD, v, -, \$t2)	12: JR \$ra	12: 0000000000000000 0000000010001010
15: (DIV, \$t1, \$t2, \$t3)	13: JUMP L1	13: 0000000000000001 1110000000000010
16: (LOAD, v, -, \$t1)	L0:	
17: (MULT, \$t3, \$t1, \$t2)	14: LOAD \$t0, u	14: 0000000000000000 0000001000000001
18: (SUB, \$t0, \$t2, \$t1)	15: LOAD \$t1, u	15: 0000000000000000 0000001010000001
19: (PARAM, \$t1, -, -)	16: LOAD \$t2, v	16: 0000000000100000 0000001100000001
20: (LOAD, v, -, \$t0)	17: DIV \$t3, \$t1, \$t2	17: 0000000000110000 1010001110011000
21: (PARAM, \$t0, -, -)	18: LOAD \$t1, v	18: 0000000000100000 0000001010000001
22: (CALL, gcd, 2, \$t0)	19: MULT \$t2, \$t3, \$t1	19: 0000000000101000 1110001100010000
23: (RET, \$t0, -, -)	20: SUB \$t1, \$t0, \$t2	20: 0000000000110000 1000001010001000
24: (LAB, L1, -, -)	21: STORE_STACK \$t1	21: 0000000000000000 0000001010010110
25: (END, gcd, -, -)	22: LOAD \$t0, v	22: 0000000000100000 0000001000000001

Continua na próxima página

Tabela 12 – Continuação da página anterior

Quádruplas	Assembly (PC)	Código Binário (PC)
26: (FUN, int, main, -)	23: STORE_STACK \$t0	23: 0000000000000000 0000001000010110
27: (IN, -, -, \$t0)	24: JAL \$ra, gcd	24: 1111111111111101 0000000010010010
28: (STORE, \$t0, x, -)	25: LOAD_STACK \$t0	25: 0000000000000000 0000001000011110
29: (IN, -, -, \$t0)	26: LOAD_STACK \$ra	26: 0000000000000000 0000000010011110
30: (STORE, \$t0, y, -)	27: STORE_STACK \$t0	27: 0000000000000000 0000001000010110
31: (LOAD, y, -, \$t0)	28: JR \$ra	28: 0000000000000000 0000000010001010
32: (PARAM, \$t0, -, -)	L1:	
33: (LOAD, x, -, \$t0)	main:	
34: (PARAM, \$t0, -, -)	29: STORE_STACK \$ra	29: 0000000000000000 0000000010010110
35: (CALL, gcd, 2, \$t0)	30: IN \$t0	30: 0000000000000000 0000001000000110
36: (OUT, \$t0, -, -)	31: STORE -, \$t0, x	31: 0000000001000000 0000001000001001
37: (END, main, -, -)	32: IN \$t0	32: 0000000000000000 0000001000000110
38: (HALT, -, -, -)	33: STORE -, \$t0, y	33: 0000000001100000 0000001000001001
	34: LOAD \$t0, y	34: 0000000001100000 0000001000000001
	35: STORE_STACK \$t0	35: 0000000000000000 0000001000010110
	36: LOAD \$t0, x	36: 0000000001000000 0000001000000001
	37: STORE_STACK \$t0	37: 0000000000000000 0000001000010110
	38: JAL \$ra, gcd	38: 1111111111111011 0100000010010010
	39: LOAD_STACK \$t0	39: 0000000000000000 0000001000011110
	40: OUT \$t0	40: 0000000000000000 0000001000001110
	41: HALT -	41: 0000000000000000 0000000000001100

5.1.3 Análise comparativa

No algoritmo de Euclides, as quádruplas de comparação e desvio representam a condição de parada da recursão. A expressão $u - u / v * v$ é decomposta em operações intermediárias de divisão, multiplicação e subtração. A chamada recursiva é representada por quádruplas de passagem de parâmetro e chamada de função, que são traduzidas para

instruções de pilha e para a instrução JAL. O retorno utiliza LOAD_STACK, STORE_STACK e JR para restaurar o endereço de retorno e disponibilizar o valor calculado.

5.2 Algoritmo de Fatorial Recursivo

O segundo caso de teste é o fatorial recursivo. Esse exemplo exercita comando condicional, chamada recursiva com um argumento, subtração, multiplicação, retorno de valor e comandos de entrada e saída.

5.2.1 Código-fonte

```
1 int fact(int n)
2 {
3     if (n <= 0) {
4         return 1;
5     }
6
7     return n * fact(n - 1);
8 }
9
10 void main(void)
11 {
12     int n;
13
14     n = input();
15     output(fact(n));
16 }
```

Listing 5.2 – Código-fonte do algoritmo de fatorial recursivo.

5.2.2 Tabela de correspondência

Tabela 13 – Tabela completa de correspondência do Algoritmo de Fatorial Recursivo

Quádruplas	Assembly (PC)	Código Binário (PC)
1: (JUMP, main, -, -)	0: JUMP main	0: 0000000000000010 1100000000000010
2: (FUN, int, fact, -)	fact:	
3: (ARG, n, -, -)	1: LOAD_STACK \$at	1: 0000000000000000 0000000110011110
4: (LOAD, n, -, \$t0)	2: STORE \$at, n, 0	2: 0000000000000000 0000000110001001
5: (LOADI, 0, -, \$t1)	3: STORE_STACK \$ra	3: 0000000000000000 0000000100101110
6: (BGT, \$t0, \$t1, L0)	4: LOAD \$t0, n	4: 0000000000000000 0000001000000001
7: (LOADI, 1, -, \$t0)	5: ADDI \$t1, \$zero, 0	5: 0000000000000000 0000001010010001
8: (RET, \$t0, -, -)	6: BGT \$t0, \$t1, L0	6: 0000000000101000 1010001000100011
9: (JUMP, L1, -, -)	7: ADDI \$t0, \$zero, 1	7: 00000000000001000 0000001000010001
10: (LAB, L0, -, -)	8: LOAD_STACK \$ra	8: 0000000000000000 0000000010011110

Continua na próxima página

Tabela 13 – Continuação da página anterior

Quádruplas	Assembly (PC)	Código Binário (PC)
11: (LAB, L1, -, -)	9: STORE_STACK \$t0	9: 0000000000000000 0000001000010110
12: (LOAD, n, -, \$t0)	10: JR \$ra	10: 0000000000000000 0000000010001010
13: (LOAD, n, -, \$t1)	11: JUMP L1	11: 0000000000000000 0000000000000010
14: (LOADI, 1, -, \$t2)	L0:	
15: (SUB, \$t1, \$t2, \$t3)	L1:	
16: (PARAM, \$t3, -, -)	12: LOAD \$t0, n	12: 0000000000000000 0000001000000001
17: (CALL, fact, 1, \$t1)	13: LOAD \$t1, n	13: 0000000000000000 0000001010000001
18: (MULT, \$t0, \$t1, \$t2)	14: ADDI \$t2, \$zero, 1	14: 0000000000001000 0000001100010001
19: (RET, \$t2, -, -)	15: SUB \$t3, \$t1, \$t2	15: 0000000000110000 1010001110001000
20: (END, fact, -, -)	16: STORE_STACK \$t3	16: 0000000000000000 0000001110010110
21: (FUN, int, main, -)	17: JAL \$ra, fact	17: 1111111111111101 1110000010010010
22: (IN, -, -, \$t0)	18: LOAD_STACK \$t1	18: 0000000000000000 0000001010011110
23: (STORE, \$t0, n, -)	19: MULT \$t2, \$t0, \$t1	19: 0000000000101000 1000001100010000
24: (LOAD, n, -, \$t0)	20: LOAD_STACK \$ra	20: 0000000000000000 0000000010011110
25: (PARAM, \$t0, -, -)	21: STORE_STACK \$t2	21: 0000000000000000 0000001100010110
26: (CALL, fact, 1, \$t0)	22: JR \$ra	22: 0000000000000000 0000000010001010
27: (OUT, \$t0, -, -)	main:	
28: (END, main, -, -)	23: STORE_STACK \$ra	23: 0000000000000000 0000000010010110
29: (HALT, -, -, -)	24: IN \$t0	24: 0000000000000000 0000001000000110
	25: STORE -, \$t0, n	25: 0000000000000000 0000001000001001
	26: LOAD \$t0, n	26: 0000000000000000 0000001000000001
	27: STORE_STACK \$t0	27: 0000000000000000 0000001000010110
	28: JAL \$ra, fact	28: 1111111111111100 1000000010010010
	29: LOAD_STACK \$t0	29: 0000000000000000 0000001000011110
	30: OUT \$t0	30: 0000000000000000 0000001000001110
	31: HALT -	31: 0000000000000000 0000000000001100

5.2.3 Análise comparativa

No fatorial recursivo, a condição $n \leq 0$ é traduzida para uma comparação seguida de desvio condicional. O caso base gera o carregamento imediato do valor 1 e o retorno da função. No caso recursivo, o compilador gera a subtração $n - 1$, empilha o argumento, realiza a chamada para **fact** e utiliza o valor retornado na multiplicação final. As instruções de pilha preservam o endereço de retorno e permitem que cada ativação da função mantenha seu próprio fluxo de execução.

5.3 Algoritmo de Ordenação: Selection Sort

O terceiro caso de teste é o algoritmo de ordenação por seleção. Esse exemplo exercita vetor global, passagem de vetor como parâmetro, acesso indexado à memória, laços de repetição, chamada de função, variáveis locais e comandos de entrada e saída.

5.3.1 Código-fonte

```
1  /* programa para ordenacao por selecao de
2     uma matriz com dez elementos. */
3
4  int vet[10];
5
6  int minloc(int a[], int low, int high)
7  {
8      int i;
9      int x;
10     int k;
11
12     k = low;
13     x = a[low];
14     i = low + 1;
15
16     while (i < high) {
17         if (a[i] < x) {
18             x = a[i];
19             k = i;
20         }
21
22         i = i + 1;
23     }
24
25     return k;
26 }
27
28 void sort(int a[], int low, int high)
29 {
30     int i;
31     int k;
32
33     i = low;
34
35     while (i < high - 1) {
36         int t;
37
38         k = minloc(a, i, high);
39         t = a[k];
40         a[k] = a[i];
41         a[i] = t;
42         i = i + 1;
43     }
44 }
45
46 void main(void)
47 {
48     int i;
49 }
```



```

50     i = 0;
51
52     while (i < 10) {
53         vet[i] = input();
54         i = i + 1;
55     }
56
57     sort(vet, 0, 10);
58
59     i = 0;
60
61     while (i < 10) {
62         output(vet[i]);
63         i = i + 1;
64     }
65 }

```

Listing 5.3 – Código-fonte do algoritmo de ordenação por seleção.

5.3.2 Tabela de correspondência

Tabela 14 – Tabela comparativa do Algoritmo de Selection Sort

Quádruplas	Assembly (PC)	Código Binário (PC)
1: (JUMP, main, -, -)	0: JUMP main	0: 0000000000001011 0100000000000010
2: (FUN, int, minloc, -)	minloc:	
3: (ARG, a, -, -)	1: LOAD_STACK \$at	1: 0000000000000000 0000000110011110
4: (ARG, low, -, -)	2: STORE \$at, a, 0	2: 0000000101000000 0000000110001001
5: (ARG, high, -, -)	3: LOAD_STACK \$at	3: 0000000000000000 0000000110011110
6: (LOAD, low, -, \$t0)	4: STORE \$at, low, 0	4: 0000000101100000 0000000110001001
7: (STORE, \$t0, k, -)	5: LOAD_STACK \$at	5: 0000000000000000 0000000110011110
8: (LOAD, low, -, \$t0)	6: STORE \$at, high, 0	6: 0000000110000000 0000000110001001
9: (LOAD, a, \$t0, \$t1)	7: STORE_STACK \$ra	7: 0000000000000000 0000000010010110
10: (STORE, \$t1, x, -)	8: LOAD \$t0, low	8: 0000000101100000 0000001000000001
11: (LOAD, low, -, \$t0)	9: STORE -, \$t0, k	9: 0000000111100000 0000001000001001
12: (LOADI, 1, -, \$t1)	10: LOAD \$t0, low	10: 0000000101100000 0000001000000001
13: (ADD, \$t0, \$t1, \$t2)	11: LOAD \$t1, a, \$t0	11: 0000000101000000 0000001010010001 12: 0000000001000000 1010001010000000 13: 0000000000000000 1010001010000001

Continua na próxima página

Tabela 14 – Continuação da página anterior

Quádruplas	Assembly (PC)	Código Binário (PC)
14: (STORE, \$t2, i, -)	14: STORE -, \$t1, x	14: 0000000111000000 0000001010001001
15: (LAB, L0, -, -)	15: LOAD \$t0, low	15: 0000000101100000 0000001000000001
16: (LOAD, i, -, \$t0)	16: ADDI \$t1, \$zero, 1	16: 0000000000001000 0000001010010001
17: (LOAD, high, -, \$t1)	17: ADD \$t2, \$t0, \$t1	17: 0000000000101000 1000001100000000
18: (BGE, \$t0, \$t1, L1)	18: STORE -, \$t2, i	18: 0000000110100000 0000001100001001
19: (LOAD, i, -, \$t0)	L0:	
20: (LOAD, a, \$t0, \$t1)	19: LOAD \$t0, i	19: 0000000110100000 0000001000000001
21: (LOAD, x, -, \$t0)	20: LOAD \$t1, high	20: 0000000110000000 0000001010000001
22: (BGE, \$t1, \$t0, L2)	21: BGE \$t0, \$t1, L1	21: 0000000010011000 1010001000101011
23: (LOAD, i, -, \$t0)	22: LOAD \$t0, i	22: 0000000110100000 0000001000000001
24: (LOAD, a, \$t0, \$t1)	23: LOAD \$t1, a, \$t0	23: 0000000101000000 0000001010010001 24: 0000000000100000 1010001010000000 25: 0000000000000000 1010001010000001
25: (STORE, \$t1, x, -)	26: LOAD \$t0, x	26: 0000000111000000 0000001000000001
26: (LOAD, i, -, \$t0)	27: BGE \$t1, \$t0, L2	27: 0000000001000000 1000001010101011
27: (STORE, \$t0, k, -)	28: LOAD \$t0, i	28: 0000000110100000 0000001000000001
28: (JUMP, L3, -, -)	29: LOAD \$t1, a, \$t0	29: 0000000101000000 0000001010010001 30: 0000000000100000 1010001010000000 31: 0000000000000000 1010001010000001
29: (LAB, L2, -, -)	32: STORE -, \$t1, x	32: 0000000111000000 0000001010001001
30: (LAB, L3, -, -)	33: LOAD \$t0, i	33: 0000000110100000 0000001000000001
31: (LOAD, i, -, \$t0)	34: STORE -, \$t0, k	34: 0000000111100000 0000001000001001
32: (LOADI, 1, -, \$t1)	35: JUMP L3	35: 0000000000000000 0000000000000010
33: (ADD, \$t0, \$t1, \$t2)	L2:	
34: (STORE, \$t2, i, -)	L3:	
35: (JUMP, L0, -, -)	36: LOAD \$t0, i	36: 0000000110100000 0000001000000001
36: (LAB, L1, -, -)	37: ADDI \$t1, \$zero, 1	37: 0000000000001000 0000001010010001
37: (LOAD, k, -, \$t0)	38: ADD \$t2, \$t0, \$t1	38: 0000000000101000 1000001100000000

Continua na próxima página

Tabela 14 – Continuação da página anterior

Quádruplas	Assembly (PC)	Código Binário (PC)
38: (RET, \$t0, -, -)	39: STORE -, \$t2, i	39: 0000000110100000 0000001100001001
39: (END, minloc, -, -)	40: JUMP L0	40: 1111111111111101 0100000000000010
40: (FUN, int, sort, -)	L1:	
41: (ARG, a, -, -)	41: LOAD \$t0, k	41: 0000000111100000 0000001000000001
42: (ARG, low, -, -)	42: LOAD_STACK \$ra	42: 0000000000000000 0000000010011110
43: (ARG, high, -, -)	43: STORE_STACK \$t0	43: 0000000000000000 0000001000010110
44: (LOAD, low, -, \$t0)	44: JR \$ra	44: 0000000000000000 000000010001010
45: (STORE, \$t0, i, -)	sort:	
46: (LAB, L4, -, -)	45: LOAD_STACK \$at	45: 0000000000000000 0000000110011110
47: (LOAD, i, -, \$t0)	46: STORE \$at, a, 0	46: 0000000101000000 0000000110001001
48: (LOAD, high, -, \$t1)	47: LOAD_STACK \$at	47: 0000000000000000 0000000110011110
49: (LOADI, 1, -, \$t2)	48: STORE \$at, low, 0	48: 0000000101100000 0000000110001001
50: (SUB, \$t1, \$t2, \$t3)	49: LOAD_STACK \$at	49: 0000000000000000 0000000110011110
51: (BGE, \$t0, \$t3, L5)	50: STORE \$at, high, 0	50: 0000000110000000 0000000110001001
52: (LOAD, high, -, \$t0)	51: STORE_STACK \$ra	51: 0000000000000000 0000000010010110
53: (PARAM, \$t0, -, -)	52: LOAD \$t0, low	52: 0000000101100000 0000001000000001
54: (LOAD, i, -, \$t0)	53: STORE -, \$t0, i	53: 0000000110100000 0000001000001001
55: (PARAM, \$t0, -, -)	L4:	
56: (LOAD, a, -, \$t0)	54: LOAD \$t0, i	54: 0000000110100000 0000001000000001
57: (PARAM, \$t0, -, -)	55: LOAD \$t1, high	55: 0000000110000000 0000001010000001
58: (CALL, minloc, 3, \$t0)	56: ADDI \$t2, \$zero, 1	56: 0000000000001000 0000001100010001
59: (STORE, \$t0, k, -)	57: SUB \$t3, \$t1, \$t2	57: 0000000000110000 1010001110001000
60: (LOAD, k, -, \$t0)	58: BGE \$t0, \$t3, L5	58: 0000000100000000 1110001000101011
61: (LOAD, a, \$t0, \$t1)	59: LOAD \$t0, high	59: 0000000110000000 0000001000000001
62: (STORE, \$t1, t, -)	60: STORE_STACK \$t0	60: 0000000000000000 0000001000010110
63: (LOAD, i, -, \$t0)	61: LOAD \$t0, i	61: 0000000110100000 0000001000000001
64: (LOAD, a, \$t0, \$t1)	62: STORE_STACK \$t0	62: 0000000000000000 0000001000010110
65: (LOAD, k, -, \$t0)	63: LOAD \$t0, a	63: 0000000101000000 0000001000000001

Continua na próxima página

Tabela 14 – Continuação da página anterior

Quádruplas	Assembly (PC)	Código Binário (PC)
66: (STORE, \$t1, a, \$t0)	64: STORE_STACK \$t0	64: 0000000000000000 0000001000010110
67: (LOAD, t, -, \$t0)	65: JAL \$ra, minloc	65: 1111111111110111 1110000010010010
68: (LOAD, i, -, \$t1)	66: LOAD_STACK \$t0	66: 0000000000000000 0000001000011110
69: (STORE, \$t0, a, \$t1)	67: STORE -, \$t0, k	67: 0000000111100000 0000001000001001
70: (LOAD, k, -, \$t0)	68: LOAD \$t0, k	68: 0000000111100000 0000001000000001
71: (LOAD, a, \$t0, \$t1)	69: LOAD \$t1, a, \$t0	69: 0000000101000000 0000001010010001 70: 0000000001000000 1010001010000000 71: 0000000000000000 1010001010000001
72: (STORE, \$t1, t, -)	72: STORE -, \$t1, t	72: 0000001000000000 0000001010001001
73: (LOAD, i, -, \$t0)	73: LOAD \$t0, i	73: 0000000110100000 0000001000000001
74: (LOAD, a, \$t0, \$t1)	74: LOAD \$t1, a, \$t0	74: 0000000101000000 0000001010010001 75: 0000000001000000 1010001010000000 76: 0000000000000000 1010001010000001
75: (LOAD, k, -, \$t0)	77: LOAD \$t0, k	77: 0000000111100000 0000001000000001
76: (STORE, \$t1, a, \$t0)	78: STORE \$t0, \$t1, a	78: 0000000101000000 0000000110010001 79: 0000000001000000 0110000110000000 80: 0000000000000000 0110001010001001
77: (LOAD, t, -, \$t0)	81: LOAD \$t0, t	81: 0000001000000000 0000001000000001
78: (LOAD, i, -, \$t1)	82: LOAD \$t1, i	82: 0000000110100000 0000001010000001
79: (STORE, \$t0, a, \$t1)	83: STORE \$t1, \$t0, a	83: 0000000101000000 0000000110010001 84: 0000000001010000 0110000110000000 85: 0000000000000000 0110001000001001
80: (LOAD, i, -, \$t0)	86: LOAD \$t0, i	86: 0000000110100000 0000001000000001
81: (LOADI, 1, -, \$t1)	87: ADDI \$t1, \$zero, 1	87: 0000000000001000 0000001010010001
82: (ADD, \$t0, \$t1, \$t2)	88: ADD \$t2, \$t0, \$t1	88: 0000000001010000 1000001100000000
83: (STORE, \$t2, i, -)	89: STORE -, \$t2, i	89: 0000000110100000 0000001100001001

Continua na próxima página

Tabela 14 – Continuação da página anterior

Quádruplas	Assembly (PC)	Código Binário (PC)
84: (JUMP, L4, -, -)	90: JUMP L4	90: 111111111111011 0110000000000010
85: (LAB, L5, -, -)	L5:	
86: (END, sort, -, -)	main:	
87: (FUN, int, main, -)	91: STORE_STACK \$ra	91: 0000000000000000 0000000010010110
88: (LOADI, 0, -, \$t0)	92: ADDI \$t0, \$zero, 0	92: 0000000000000000 0000001000010001
89: (STORE, \$t0, i, -)	93: STORE -, \$t0, i	93: 0000000110100000 0000001000001001
90: (LAB, L6, -, -)	L6:	
91: (LOAD, i, -, \$t0)	94: LOAD \$t0, i	94: 0000000110100000 0000001000000001
92: (LOADI, 10, -, \$t1)	95: ADDI \$t1, \$zero, 10	95: 0000000001010000 0000001010010001
93: (BGE, \$t0, \$t1, L7)	96: BGE \$t0, \$t1, L7	96: 0000000001010000 1010001000101011
94: (IN, -, -, \$t0)	97: IN \$t0	97: 0000000000000000 0000001000000110
95: (LOAD, i, -, \$t1)	98: LOAD \$t1, i	98: 0000000110100000 0000001010000001
96: (STORE, \$t0, vet, \$t1)	99: STORE \$t1, \$t0, vet	99: 0000000000000000 0000000110010001 100: 00000000000101000 0110000110000000 101: 0000000000000000 0110001000001001
97: (LOAD, i, -, \$t0)	102: LOAD \$t0, i	102: 0000000110100000 0000001000000001
98: (LOADI, 1, -, \$t1)	103: ADDI \$t1, \$zero, 1	103: 00000000000001000 0000001010010001
99: (ADD, \$t0, \$t1, \$t2)	104: ADD \$t2, \$t0, \$t1	104: 00000000000101000 1000001100000000
100: (STORE, \$t2, i, -)	105: STORE -, \$t2, i	105: 0000000110100000 0000001100001001
101: (JUMP, L6, -, -)	106: JUMP L6	106: 1111111111111110 0110000000000010
102: (LAB, L7, -, -)	L7:	
103: (LOADI, 10, -, \$t0)	107: ADDI \$t0, \$zero, 10	107: 0000000001010000 0000001000010001
104: (PARAM, \$t0, -, -)	108: STORE_STACK \$t0	108: 0000000000000000 0000001000010110
105: (LOADI, 0, -, \$t0)	109: ADDI \$t0, \$zero, 0	109: 0000000000000000 0000001000010001
106: (PARAM, \$t0, -, -)	110: STORE_STACK \$t0	110: 0000000000000000 0000001000010110
107: (LOAD, vet, -, \$t0)	111: LOAD \$t0, vet	111: 0000000000000000 0000001000000001
108: (PARAM, \$t0, -, -)	112: STORE_STACK \$t0	112: 0000000000000000 0000001000010110
109: (CALL, sort, 3, \$t0)	113: JAL \$ra, sort	113: 1111111111110111 0110000010010010

Continua na próxima página

Tabela 14 – Continuação da página anterior

Quádruplas	Assembly (PC)	Código Binário (PC)
110: (LOADI, 0, -, \$t1)	114: LOAD_STACK \$t0	114: 0000000000000000 0000001000011110
111: (STORE, \$t1, i, -)	115: ADDI \$t1, \$zero, 0	115: 0000000000000000 0000001010010001
112: (LAB, L8, -, -)	116: STORE -, \$t1, i	116: 0000000110100000 0000001010001001
113: (LOAD, i, -, \$t1)	L8:	
114: (LOADI, 10, -, \$t2)	117: LOAD \$t1, i	117: 0000000110100000 0000001010000001
115: (BGE, \$t1, \$t2, L9)	118: ADDI \$t2, \$zero, 10	118: 0000000001010000 0000001100010001
116: (LOAD, i, -, \$t1)	119: BGE \$t1, \$t2, L9	119: 0000000001010000 1100001010101011
117: (LOAD, vet, \$t1, \$t2)	120: LOAD \$t1, i	120: 0000000110100000 0000001010000001
118: (OUT, \$t2, -, -)	121: LOAD \$t2, vet, \$t1	121: 0000000000000000 0000001100010001 122: 0000000000101000 1100001100000000 123: 0000000000000000 1100001100000001
119: (LOAD, i, -, \$t1)	124: OUT \$t2	124: 0000000000000000 0000001100001110
120: (LOADI, 1, -, \$t2)	125: LOAD \$t1, i	125: 0000000110100000 0000001010000001
121: (ADD, \$t1, \$t2, \$t3)	126: ADDI \$t2, \$zero, 1	126: 00000000000001000 0000001100010001
122: (STORE, \$t3, i, -)	127: ADD \$t3, \$t1, \$t2	127: 0000000000110000 1010001110000000
123: (JUMP, L8, -, -)	128: STORE -, \$t3, i	128: 0000000110100000 0000001110001001
124: (LAB, L9, -, -)	129: JUMP L8	129: 1111111111111110 0110000000000010
125: (END, main, -, -)	L9:	
126: (HALT, -, -, -)	130: HALT -	130: 0000000000000000 0000000000001100

5.3.3 Análise comparativa

No algoritmo de ordenação, as operações sobre vetores exigem tratamento adicional na etapa de geração de código. Quádruplas como (LOAD, a, \$t0, \$t1) e (STORE, \$t1, a, \$t0) representam acessos indexados. Na tradução para assembly, esses acessos podem ser expandidos em mais de uma instrução binária, pois o endereço efetivo precisa ser calculado a partir do endereço-base do vetor e do índice. Quando o vetor é recebido como parâmetro, o valor associado ao identificador representa o endereço-base utilizado para acessar os elementos.

5.4 Síntese dos Resultados

Os três testes cobrem os principais recursos implementados no compilador. O exemplo do máximo divisor comum valida chamadas recursivas com múltiplos argumentos e expressões aritméticas compostas. O exemplo do fatorial valida recursão simples, caso base e propagação do valor de retorno. O exemplo de ordenação valida laços, vetores, passagem de vetor como parâmetro e expansão de acessos indexados para instruções executáveis no processador-alvo.

6 Conclusão

O projeto desenvolvido implementa um compilador para a linguagem C-, tendo como arquitetura-alvo um processador autoral baseado na filosofia RISC. O fluxo de compilação contempla as etapas de análise léxica, análise sintática, construção da árvore sintática abstrata, análise semântica, geração de código intermediário em quádruplas, geração de código assembly e geração do código binário executável para o processador.

A implementação permitiu estabelecer a correspondência entre construções da linguagem C- e instruções da arquitetura-alvo. Foram tratados comandos condicionais, laços de repetição, atribuições, expressões aritméticas e relacionais, chamadas de função, recursão, vetores, comandos de entrada e saída e retorno de valores. Dessa forma, o compilador realiza a tradução de programas em C- para uma sequência de instruções compatível com a organização de memória, registradores, pilha e conjunto de instruções do processador.

6.1 Dificuldades Encontradas

Uma das principais dificuldades foi o mapeamento entre a árvore sintática abstrata e o código intermediário. Como diferentes nós da árvore representam construções com comportamentos distintos, foi necessário definir regras específicas para expressões, comandos condicionais, laços, chamadas de função, retornos e acessos a vetores. Esse mapeamento também exigiu controle sobre registradores temporários, rótulos e valores intermediários.

Outra dificuldade esteve relacionada à tradução das quádruplas para assembly. Algumas quádruplas possuem correspondência direta com uma instrução da arquitetura, enquanto outras exigem a geração de múltiplas instruções. Esse caso ocorre principalmente em chamadas de função, retornos, passagem de parâmetros, salvamento de contexto e acesso indexado a vetores. Também foi necessário tratar corretamente os rótulos e os deslocamentos usados em desvios condicionais e incondicionais.

A geração do código binário exigiu alinhamento entre o assembly produzido e os formatos de instrução definidos para o processador. Cada instrução precisou ser codificada de acordo com seus campos específicos, como opcode, funct, registradores, imediato e deslocamento. Esse processo demandou cuidado para preservar a correspondência entre o endereço da instrução no assembly e a palavra binária gerada.

Também houve desafios no gerenciamento de memória. O compilador precisou organizar variáveis globais, variáveis locais, parâmetros, vetores e pilha de execução. O suporte a chamadas recursivas exigiu o uso da pilha para armazenar argumentos, valores de

retorno e endereço de retorno, evitando a perda de informações entre ativações sucessivas de uma mesma função.

6.2 Destaques

O compilador implementado realiza o fluxo completo de tradução, desde o código-fonte em C- até o código binário executável pelo processador. A geração de uma representação intermediária em quádruplas facilitou a separação entre a análise do programa-fonte e a geração de código para a arquitetura-alvo.

A análise semântica utiliza tabela de símbolos para registrar identificadores, funções, parâmetros e informações de tipo. Essa etapa permite detectar erros como uso de identificadores não declarados, conflitos de declaração, uso inadequado de funções e inconsistências associadas a tipos e retornos.

A geração de código contempla construções relevantes da linguagem C-, incluindo funções recursivas, passagem de parâmetros, retorno de valores, vetores e comandos de entrada e saída. Os exemplos apresentados demonstram a aplicação do compilador em programas com diferentes características, como o cálculo do máximo divisor comum, o fatorial recursivo e a ordenação por seleção.

Outro ponto relevante é a integração entre o compilador e o processador. O código assembly gerado é convertido para palavras binárias de 32 bits, respeitando os formatos de instrução da arquitetura. Com isso, o resultado final pode ser carregado na memória de instruções do processador para execução.

6.3 Trabalhos Futuros

Como trabalhos futuros, pode-se ampliar a análise semântica para tratar escopos de bloco de forma mais completa e verificar caminhos de retorno em funções com maior precisão. Também é possível aprimorar as mensagens de erro, incluindo informações mais detalhadas sobre linha, contexto e tipo de inconsistência encontrada.

Outra extensão possível é a implementação de otimizações no código intermediário e no código assembly, como eliminação de instruções redundantes, melhor reutilização de registradores temporários e simplificação de expressões constantes. Essas otimizações poderiam reduzir o número de instruções geradas e melhorar o uso da memória de instruções.

Também pode ser desenvolvido um modelo mais estruturado de frame de ativação para chamadas de função, separando de forma mais explícita parâmetros, variáveis locais, endereço de retorno e valores temporários. Essa alteração facilitaria a expansão do

compilador para programas maiores e com maior aninhamento de chamadas.

Por fim, o compilador pode ser estendido para suportar novos recursos da linguagem, como operadores adicionais, novos tipos de dados, escopos mais refinados e mecanismos mais completos de inicialização de variáveis e vetores. Também é possível automatizar uma suíte de testes para comparar código-fonte, quádruplas, assembly e binário gerados em diferentes programas de entrada.

Referências

AHO, A. V. et al. *Compilers: Principles, Techniques, and Tools*. 2. ed. Boston: Pearson, 2007. Citado 2 vezes nas páginas 8 e 34.

Free Software Foundation. *Bison: The Yacc-compatible Parser Generator*. [S.l.], 2024. Disponível em: <<https://www.gnu.org/software/bison/>>. Citado na página 23.

LOUDEN, K. C.; SILVA, F. S. C. d. *Compiladores: princípios e práticas*. São Paulo: Pioneira Thomson Learning, 2004. Citado 2 vezes nas páginas 8 e 21.

PATTERSON, D.; WATERMAN, A. *Guia Prático RISC-V: Atlas de uma Arquitetura Aberta*. 1.0.0. ed. Berkeley, California: Strawberry Canyon LLC, 2019. Tradução para o português por Luiz Gustavo Xavier, Nathan Formentin e Marcelo Pias. ISBN 978-0-9992491-1-6. Citado na página 10.

PATTERSON, D. A.; HENNESSY, J. L. *Organização e projeto de computadores: a interface hardware/software*. 5. ed. Rio de Janeiro: GEN LTC, 2017. Citado na página 10.